SOFTWARE ARCHITECTURE AND DESIGN DOCUMENT

Hostel Buddy

Smart Hostel Complaint Management System

Version 1.0

Prepared by: Group 19

Pranjal Tyagi · Dinesh Saini · Ajinkya · Saurabh · Vijay

Department of Computer Science and Engineering

IIIT Kottayam

5 August 2026

Revision History

Name	Date	Reason for Change	Version
Group 19	5 August 2026	Initial release of the Software Architecture and Design Document, derived from the approved SRS v1.0.	1.0

Group 19 — Team Members

#	Member Name	Roll Number
1	Pranjal Tyagi	2024BCD0053
2	Dinesh Saini	2024BCD0033
3	Ajinkya	2024BCS0173
4	Saurabh	2024BCS0185
5	Vijay	2024BCS0249

Table of Contents

1. Introduction

1.1 Purpose

This Software Architecture and Design Document (SADD) describes **how** the Hostel Buddy system is designed and implemented. It is the direct successor to the approved *Software Requirements Specification (SRS) v1.0*, which defined **what** the system must do.

Where the SRS states requirements in terms of externally observable behaviour (numbered `REQ-1`…`REQ-41`), this document translates each of those requirements into concrete architectural decisions: the layers of the system, the software modules and their responsibilities, the database schema, the classes and their interactions, and the deployment topology. Section 12 provides a traceability matrix proving that every functional requirement in the SRS is realised by at least one design element.

The document also records the *reasoning* behind each decision through the Design Rationale sections, so that a future maintainer can understand not only the shape of the system but why it took that shape.

1.2 Scope

Hostel Buddy is a three-tier web application that replaces a hostel's paper complaint register with a digital, searchable and trackable platform. Students register, raise maintenance complaints under a fixed set of categories with an optional photograph, and follow those complaints through the lifecycle *Pending → In Progress → Resolved → Closed*. A single administrator reviews every complaint, searches and filters them, advances their status and records remarks, and views analytics summarising complaint activity.

This document covers the complete design of Version 1.0: the presentation, application and data tiers; all four functional modules (Authentication, User Management, Complaint Management, Dashboard & Analytics); the supporting cross-cutting infrastructure (security, upload, error handling); and the single-node deployment model.

Out of scope (deferred, per SRS §8): email and push notifications, a separate maintenance-staff role, complaint ratings, QR-code room identification, a native mobile application, AI-based categorisation and multi-hostel support. The architecture nevertheless leaves defined extension points for these, described in §13.3.

1.3 Intended Audience

Reader	How to use this document
Developers	Primary implementation reference. Read §4 (modules), §6 (database), §7 (classes and interactions) before writing code.
Testers	Use §12 (traceability matrix) to derive test cases, and §7.4 sequence diagrams to understand expected message flows and failure paths.
Project Guide / Evaluators	Read §1–§3 for context, then the Design Rationale boxes throughout, which justify every significant decision.
Maintainers	Read §2.3 (design principles), §4.3 (decomposition) and Appendix D (design decisions) to understand where a change belongs and what it will affect.

1.4 References

#	Title	Source / Version
R1	Hostel Buddy — Software Requirements Specification	Group 19, v1.0 (approved)
R2	IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications	IEEE, 1998
R3	IEEE Std 1016-2009 — Standard for Information Technology, Systems Design, Software Design Descriptions	IEEE, 2009
R4	Unified Modeling Language (UML) Specification	OMG UML 2.5.1
R5	Express.js Framework Documentation	Express 4.x
R6	Node.js API Documentation — `node:sqlite` module	Node.js 22.5+ / 24.x
R7	RFC 7519 — JSON Web Token (JWT)	IETF, 2015
R8	OWASP Top 10 — Web Application Security Risks	OWASP, 2021

1.5 Definitions and Acronyms

Term	Definition
SADD	Software Architecture and Design Document — this document.
SRS	Software Requirements Specification — the approved requirements baseline (R1).
API	Application Programming Interface; here, the JSON/HTTP interface exposed by the application tier.
REST	Representational State Transfer — the architectural style of the HTTP API.
JWT	JSON Web Token — a signed, self-contained token carrying the user's identity and role.
RBAC	Role-Based Access Control — permissions granted according to the user's role.
DAL / Repository	Data Access Layer — the only layer permitted to execute SQL.
DTO	Data Transfer Object — a plain object carrying data across a layer boundary.
CRUD	Create, Read, Update, Delete.
Middleware	A function in the Express pipeline that processes a request before it reaches a controller.
CSP	Content Security Policy — an HTTP header restricting which resources a page may load.
WAL	Write-Ahead Logging — a SQLite journalling mode improving read concurrency.
bcrypt	An adaptive password-hashing function with a configurable work factor.
Complaint Lifecycle	The fixed state sequence Pending → In Progress → Resolved → Closed.

2. Architectural Overview

2.1 Architectural Goals

The architecture is shaped by the non-functional requirements of the SRS (`NFR-1`…`NFR-13`). Each goal below states the quality attribute, the driving requirement and the architectural mechanism that delivers it.

Goal	Driven by	Architectural mechanism
Security	NFR-8… NFR-13	bcrypt password hashing; stateless JWT authentication; role guards at the route layer plus ownership checks in the service layer; server-side validation; security headers and a strict Content Security Policy.
Maintainability	NFR (Quality)	Four independent feature modules, each internally split into routes → controller → service → repository. A change to persistence touches only repositories.
Scalability	NFR-4	A stateless application tier (no server-side session store) so instances can be replicated behind a load balancer; indexed, paginated queries; aggregation pushed into SQL.
Performance	NFR-1…NFR-3	Indexes on the three hot query paths; `GROUP BY` aggregation instead of in-memory counting; bounded page sizes; static assets served directly.
Reliability	NFR-5…NFR-7	Durable relational storage with WAL journalling; database-level `CHECK` constraints as a data-integrity backstop; one central error handler so no failure escapes unhandled.
Portability	NFR (Quality)	Browser-based client requiring no installation; a database driver built into the runtime, so the system builds without a native compiler toolchain.
Testability	—	Business rules concentrated in services (callable without HTTP) and a documented REST contract enabling black-box integration tests.

2.2 Architectural Style

2.2.1 Selected Style

Hostel Buddy adopts a **three-tier client–server architecture** whose application tier is internally organised as a **layered (n-tier) architecture**, exposing a **RESTful API** consumed by a thin browser client.

Reason for Selection

- **The problem is data-centric and CRUD-dominated.** The system's essential complexity is validating, storing, retrieving and reporting on complaint records — precisely the shape a layered architecture handles best.
- **Requirements demand a strict authority boundary.** `REQ-18`/`REQ-19` (edit only while Pending) and `REQ-20` (no cross-student access) must hold regardless of which client calls. A layered design lets these invariants live in one server-side layer that every path must traverse.
- **Two distinct clients share one behaviour set.** Students and the administrator use different screens but the same rules; a REST API with role-based authorisation serves both without duplicating logic.
- **Team scale and skills.** A four-member team benefits from a familiar, well-documented structure with clear file-level ownership rather than a distributed or event-driven design.

Advantages

- **Separation of concerns** — each layer has exactly one reason to change; SQL exists in one layer only.
- **Substitutability** — the database can be replaced (SQLite → PostgreSQL) by rewriting repositories alone; the browser client could be replaced by a mobile app with no server change.
- **Security by construction** — every request passes the same authentication and authorisation middleware; rules cannot be bypassed by adding a new route.
- **Testability** — services are plain functions independent of HTTP, and the API is testable end-to-end.
- **Horizontal scalability** — statelessness removes server affinity.

Limitations

- **Layer traversal overhead** — a trivial read still passes route → controller → service → repository. Accepted: the indirection is a few function calls, negligible against I/O.
- **Some boilerplate** — each module repeats a four-file pattern. Accepted as the cost of uniformity; it makes the codebase predictable.
- **Single deployable unit** — the whole application scales as one. Accepted for v1; modules are cleanly separated, so extraction into services is possible later.
- **No independent module deployment** — unlike microservices, a change to any module requires redeploying the application. Acceptable at this scale and release cadence.

Architecture Diagram

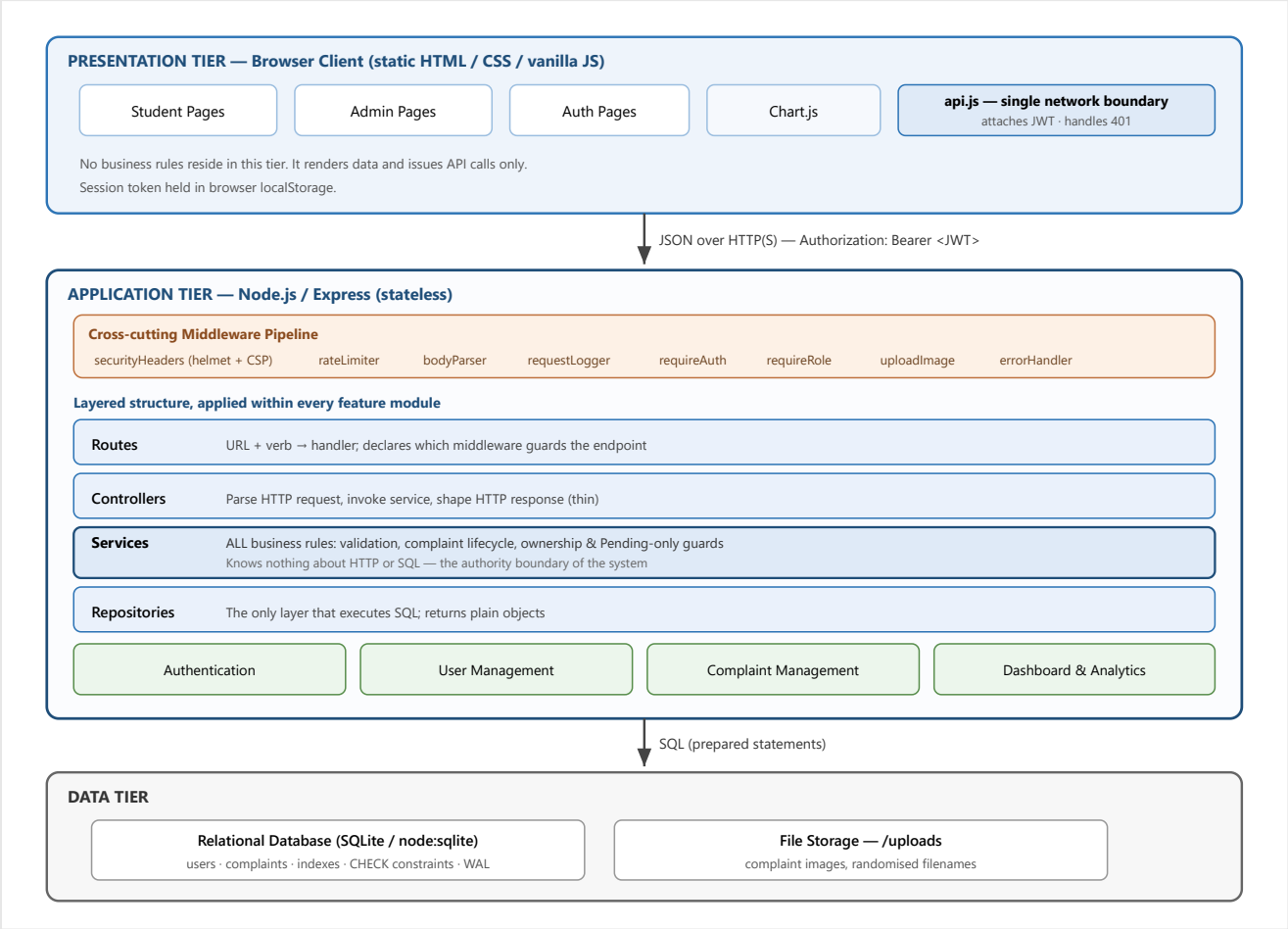

Figure 2.1 — Layered three-tier architecture of Hostel Buddy.

> **Design Rationale.** A layered architecture was chosen over the two realistic alternatives. A *monolithic MVC application with server-rendered pages* would have been marginally faster to build but would couple presentation to business logic, making the planned mobile client (§13.3) impossible without rewriting the server. A *microservice architecture* would provide independent deployability the project does not need, while adding network partitioning, distributed transactions and operational overhead that a four-person team cannot justify for a single hostel. The layered monolith with a REST boundary captures the main benefit of both — a reusable, client-agnostic API with enforced internal separation — at the cost of only per-module boilerplate. The single most important consequence is that **business rules live in the service layer**: because `REQ-18`–`REQ-20` are invariants rather than transport concerns, placing them in services guarantees they hold for every caller, present or future.

2.3 Design Principles

Principle	Application in Hostel Buddy
Modularity	The system is decomposed into four feature modules aligned with major functional responsibilities (not with screens or tables). Each is self-contained in its own directory with a uniform internal structure.
High cohesion	Every element inside a module serves one responsibility: the Complaint module owns complaint creation, editing, lifecycle transitions and retrieval — and nothing else.

Low coupling	Modules interact only through published service functions, never by reaching into another module's repository or database tables. The Dashboard module, for example, composes counts exposed by the Complaint and User repositories rather than issuing its own joins across their tables.
Separation of concerns	Transport (routes/controllers), policy (services) and persistence (repositories) are strictly separated. A controller contains no rule; a service contains no SQL.
Single responsibility	Each file has one reason to change. `jwt.js` changes only if the token scheme changes; `upload.js` only if file handling changes.
Defence in depth	Critical rules are enforced at more than one level: role guards at routes *and* ownership checks in services; enum validation in services *and* `CHECK` constraints in the schema.
Don't repeat yourself (DRY)	Cross-cutting behaviour is centralised: one error handler, one authentication middleware, one client-side network wrapper, one source of truth for categories/statuses/roles (`constants.js`).
Reuse	The same REST API serves both user roles and would serve a future mobile client unchanged. Shared helpers (validators, JWT, UI utilities) are used across modules and pages.
Fail fast, fail safe	Invalid input is rejected at the earliest layer that can judge it; unexpected errors are converted into a uniform response that never leaks internals; the server refuses to start in production with insecure configuration.
Configuration over hard-coding	Ports, secrets, credentials and paths come from environment variables via a single typed configuration module; no secret appears in source.

2.4 Technology Stack

Layer	Technology	Reason for selection
Presentation	HTML5, CSS3, vanilla JavaScript (ES2020)	No build step or framework runtime; the UI is small and form-driven. Keeps the client trivially deployable and readable by every team member.
Charting	Chart.js 4 (served locally)	Satisfies **REQ-36** (category distribution chart) with minimal code. Vendored rather than loaded from a CDN so the demo works offline and the strict CSP needs no external origin.
Application runtime	Node.js 22.5+ (24.x used)	Single language across client and server; mature HTTP ecosystem; ships the database driver in the runtime itself.
Web framework	Express 4	Minimal and unopinionated, which allows the explicit layering in §2.2 rather than imposing a competing structure. Its middleware pipeline maps directly onto the cross-cutting concerns.
Database	SQLite via built-in `node:sqlite`	Real SQL with constraints and indexes, zero configuration, single-file storage. Being built into Node it requires **no native compilation**, so any team member can clone and run without a C++ toolchain. Schema is written to remain PostgreSQL-compatible.
Authentication	`jsonwebtoken` (JWT, HS256)	Stateless tokens remove the session store, directly serving the scalability goal (**NFR-4**).
Password hashing	`bcryptjs` (cost factor 10)	Adaptive, salted hashing as required by **NFR-8**; pure-JS implementation avoids native builds.
File upload	`multer` 2.x	Standard multipart handling for **REQ-12**, with built-in MIME-type filtering and size limits.
HTTP security	`helmet` , `express-rate-limit`	Security headers and a Content Security Policy (**NFR-11**/**NFR-12**); brute-force mitigation on authentication endpoints.
Configuration	`dotenv`	Keeps secrets out of source control and makes environment promotion (dev → production) a configuration change only.
Version control	Git / GitHub	Phase-wise commit history documenting the incremental construction of the system.

2.5 Design Rationale

The following table summarises the architecture-level decisions, the alternatives considered and the trade-offs consciously accepted. Detailed records are in Appendix D.

Decision	Alternative considered	Justification and accepted trade-off
Layered monolith with REST API	Server-rendered MVC; microservices	Gives a client-agnostic API and enforced internal separation without distributed-system complexity. Trade-off: the whole application scales as a unit.
Business rules in the service layer	Rules in route handlers or controllers	A route-level check protects only that route; a service-level invariant holds for every caller. Trade-off: a small amount of indirection.
Stateless JWT authentication	Server-side sessions with a session store	Removes shared state so instances replicate freely (`NFR-4`). Trade-off: a token cannot be revoked before expiry; mitigated by a 24-hour lifetime.
Relational database with enum `CHECK` constraints	Document store; application-only validation	The data is highly structured with a fixed vocabulary; constraints keep data valid even if application code is bypassed by a script. Trade-off: schema changes require migration.
Built-in `node:sqlite` driver	`better-sqlite3` (native module)	Eliminates the native build step that fails without Visual Studio build tools, so every team member can run the project. Trade-off: the module is marked experimental; isolation in the repository layer makes replacement cheap.
Aggregation performed in SQL	Fetch rows and count in JavaScript	Keeps payloads small and uses the engine best suited to counting; measured at ~1 ms for the full admin dashboard over 2,000 complaints. Trade-off: logic expressed in SQL rather than application code.
Local filesystem for images	Cloud object storage (S3)	Simplest working solution for a single-node demo. Trade-off: images are not shared across replicas — documented as the first change required when scaling out.
Single administrator role (no maintenance staff)	Three-role model with work assignment	Preserves the complete complaint workflow at a fraction of the complexity, as scoped by the SRS. Trade-off: no per-technician assignment or workload tracking.

3. System Context

3.1 Context Diagram

The context diagram establishes the system boundary: what lies inside Hostel Buddy, which external actors interact with it, and what information crosses the boundary.

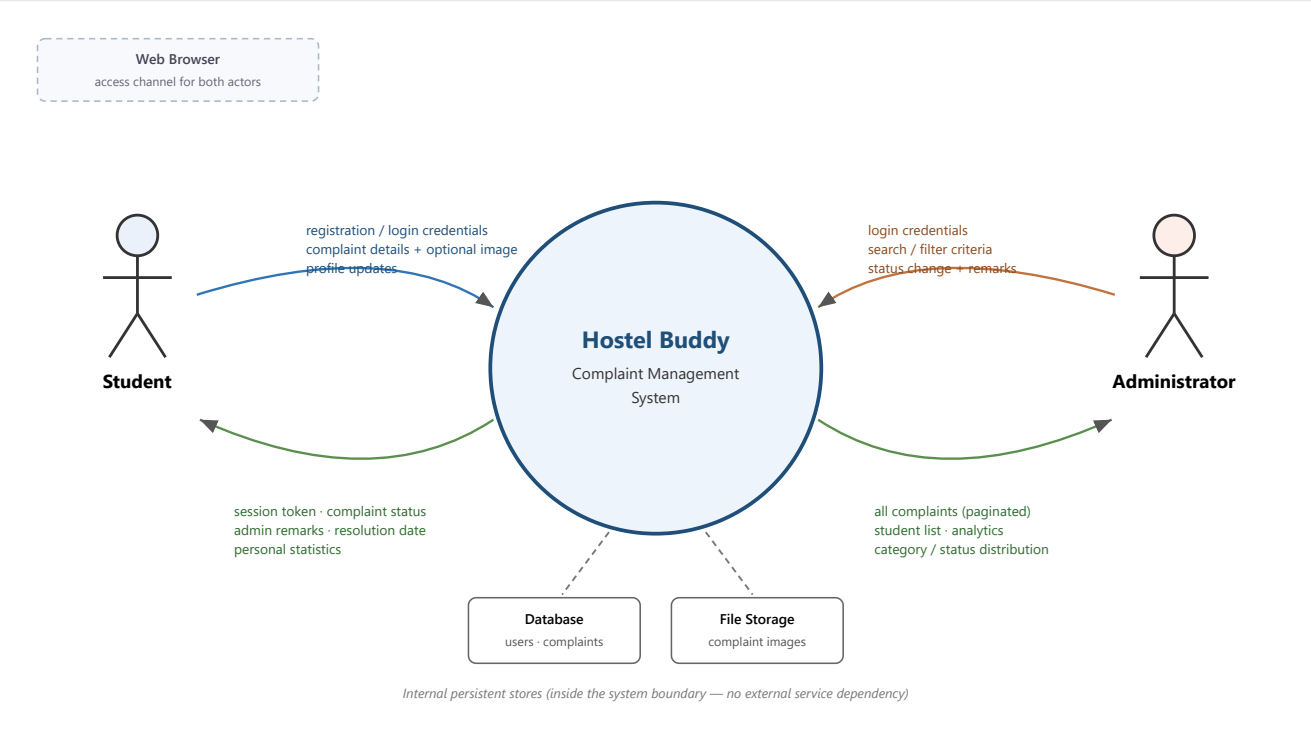

Figure 3.1 — System Context Diagram.

> **Design Rationale.** The boundary is drawn to make Hostel Buddy **self-contained**: both the database and image storage sit inside it, and there is no dependency on any third-party service. This is deliberate — the SRS scopes v1.0 to a single hostel with notifications explicitly deferred, so introducing an external mail gateway or object store would add failure modes and credentials to manage without serving any current requirement. Only two human actors cross the boundary, and they are distinguished solely by role, which is why a single API with role-based authorisation is sufficient rather than two separate interfaces. The browser is shown as the access channel rather than an actor, because it carries no application logic of its own.

3.2 External Systems

Hostel Buddy v1.0 integrates with **no external systems, third-party APIs or paid services**. All processing and storage occur within the deployed application and its own data stores. The only components outside the application process are infrastructure the system depends on rather than services it calls:

External component	Type	Interaction
Web browser	Client platform	Renders the interface; issues HTTP requests; stores the session token in `localStorage` ; supplies image files from device storage or camera through the file-picker.

Host operating system / filesystem	Platform service	Persists the database file and uploaded images.
Network / TLS terminator	Infrastructure	Transports requests; in deployment, terminates HTTPS in front of the application (**NFR-12**).

Planned future integrations — an SMTP gateway for email notification and a push-notification service — are identified in §13.3 together with the extension point at which they attach (a hook in the complaint service after a status transition).

3.3 High-Level Data Flow

The diagram below traces how information moves from user input, through validation and business rules, into persistent storage, and back out as reports and status views.

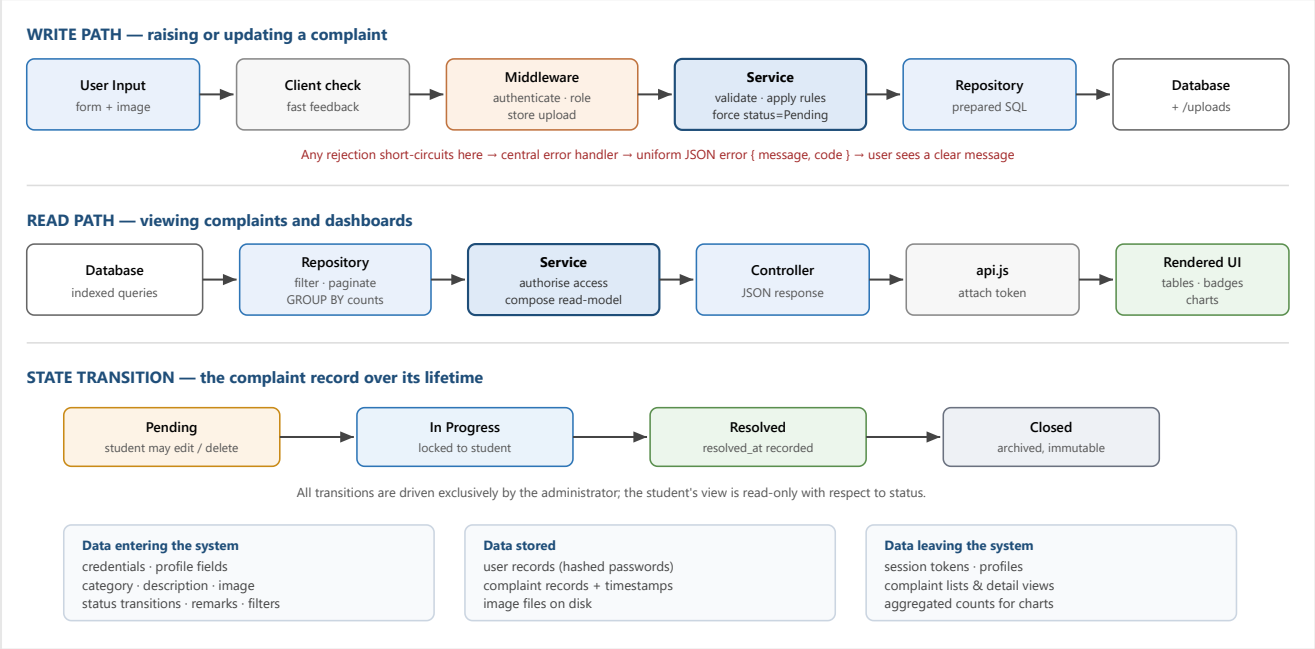

Figure 3.2 — High-level data flow: write path, read path and record lifecycle.

Three properties of this flow are architecturally significant. First, **the service layer is on every path** — no request reaches the database without passing the layer that owns the rules. Second, **validation is duplicated deliberately**: the client checks for responsiveness, but the server re-validates because the client is never trusted. Third, **aggregation happens in the data tier**, so dashboard responses carry a handful of numbers rather than thousands of rows.

4. Module Identification and Design

4.1 Module Identification

System functionality was grouped into modules by **major functional responsibility**, not by screen or by database table. The seven system features of the SRS (§4.1–§4.7) were examined for shared data, shared rules and common reasons to change; features that manipulate the same core concept under the same rules were collapsed into a single module.

Criteria applied

Criterion	How it was applied
Functional cohesion	Every element of a module contributes to one responsibility. Complaint submission, editing, deletion, retrieval and lifecycle transitions all operate on the complaint concept under one consistent rule set, so they form one module rather than four.
Low coupling	Modules communicate only through published service functions. No module reads another module's table directly, so a change to one module's storage cannot silently break another.
Single reason to change	Authentication is separated from User Management because credential policy (hashing, token lifetime) evolves independently of profile data.
Separation of concerns	Cross-cutting behaviour used by every module — authentication checks, file upload, error translation, security headers — was extracted into a shared infrastructure layer instead of being duplicated.
Maintainability	Each module is a self-contained directory with an identical internal structure, so a developer can locate any behaviour without a map of the codebase.
Read/write asymmetry	Dashboard & Analytics is a distinct module because it is a read-only projection over data owned by others; separating it keeps reporting queries from complicating transactional logic.

Identified modules

ID	Module	Primary responsibility	SRS features covered
M1	Authentication Module	Establishes and verifies identity: registration, login, password hashing, token issue, and the guards that protect every other endpoint.	§4.1 (partly)
M2	User Management Module	Owns the user record after identity is established: profile retrieval and update, and the administrator's view of registered students.	§4.1 (partly), §4.7
M3	Complaint Management Module	Owns the complaint concept end to end: submission with image, retrieval, student edit/delete, administrator search and filtering, and lifecycle transitions with remarks.	§4.2, §4.3, §4.4, §4.5
M4	Dashboard & Analytics Module	Read-only reporting: composes per-student and system-wide statistics and category/status distributions for visualisation.	§4.6
M5	Infrastructure & Cross-Cutting Layer	Shared services consumed by all modules: configuration, database connection and schema, security middleware, upload handling, error translation, logging, JWT and validation utilities.	Supports all; realises NFRs

Deliberately avoided: a module per screen (e.g. a "Login Page module"), a module per table (e.g. a "Users table module"), and a "utility" grab-bag. M5 is not a grab-bag — it contains only genuinely cross-cutting concerns, each in its

own single-responsibility file.

4.2 Functional Requirement to Module Mapping

Every functional requirement in the SRS is implemented by at least one module. Requirements are grouped by their SRS feature for readability; the full requirement-to-class-to-test mapping appears in §12.

SRS Feature	Functional Requirement	Module Responsible	Supporting modules
§4.1 User Authentication & Account Management	**REQ-1** Register student account **REQ-2** Unique email enforcement **REQ-3** Protected password storage **REQ-4** Authenticate and establish session **REQ-5** Generic invalid-login error **REQ-6** Single seeded administrator **REQ-7** Logout / session termination **REQ-8** Validate credentials input	**M1 — Authentication**	M2 (user record persistence), M5 (JWT, hashing, validation, seeding)
§4.2 Complaint Submission	**REQ-9** Submit complaint **REQ-10** Restrict to defined categories **REQ-11** Require description **REQ-12** Optional image within limits **REQ-13** Initial state Pending + timestamp **REQ-14** Associate with submitting student **REQ-15** Confirm submission	**M3 — Complaint Management**	M1 (identity of submitter), M5 (upload middleware, constants)
§4.3 Complaint Tracking & History	**REQ-16** List own complaints **REQ-17** Display status, dates, remarks **REQ-18** Edit only while Pending **REQ-19** Delete only while Pending **REQ-20** No access to others' complaints **REQ-21** Reflect administrator changes	**M3 — Complaint Management**	M1 (authenticated identity for ownership checks)
§4.4 Complaint Management (Administrator)	**REQ-22** View all complaints **REQ-23** Change status within lifecycle **REQ-24** Add/update remarks **REQ-25** Record last-change time **REQ-26** Record resolution date once **REQ-27** Restrict transitions to administrator **REQ-28** Reject undefined states	**M3 — Complaint Management**	M1 (role guard), M2 (student details in listings), M5 (constants)
§4.5 Search & Filter	**REQ-29** Search by ID, name, room **REQ-30** Filter by category		

	REQ-31 Filter by status **REQ-32** Combine criteria	**M3 — Complaint Management**	M2 (joined student name/room), M1 (role guard)
§4.6 Dashboard & Analytics	**REQ-33** Student statistics **REQ-34** Total students and complaints **REQ-35** Counts per status **REQ-36** Category distribution chart **REQ-37** Recent complaints list	**M4 — Dashboard & Analytics**	M3 (complaint aggregates), M2 (student count), M1 (role guard)
§4.7 Profile & Student Account Management	**REQ-38** View/update own profile **REQ-39** Validate profile changes **REQ-40** Administrator views student list **REQ-41** Administrator manages accounts	**M2 — User Management**	M1 (authenticated identity, role guard), M5 (validation)

Coverage check: requirements **REQ-1** through **REQ-41** are each assigned to exactly one owning module, with supporting modules listed where collaboration is required. No requirement is unassigned, and no module exists without at least one requirement to justify it.

4.3 Module Decomposition

Figure 4.1 decomposes the system into its modules and shows the internal four-layer structure repeated within each feature module, together with the shared infrastructure they all depend on.

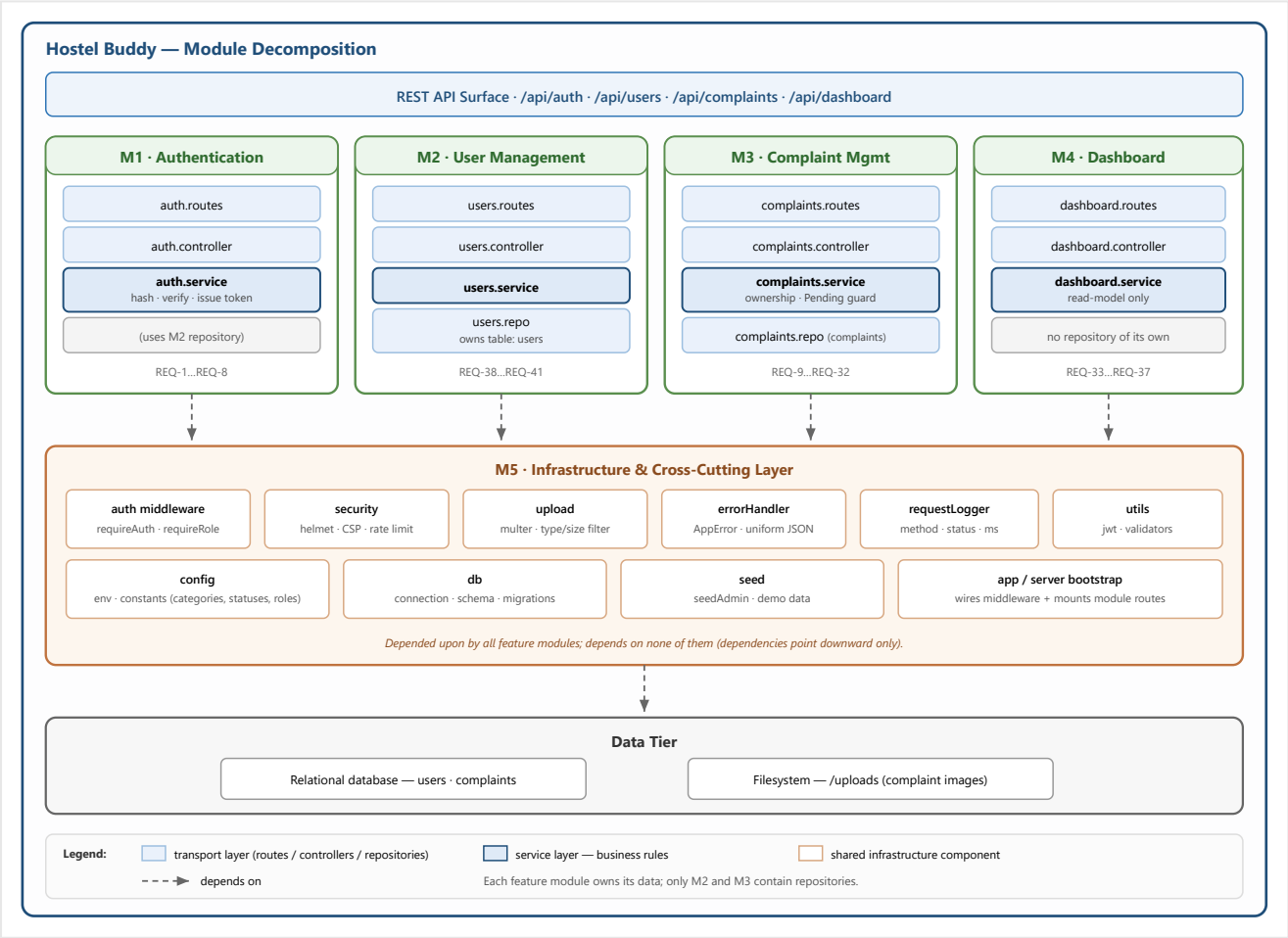

Figure 4.1 — Module Decomposition Diagram.

> **Design Rationale.** Three decisions in this decomposition deserve justification. **(1) Authentication (M1) is separate from User Management (M2)** even though both concern users, because they change for different reasons — token lifetime and hashing policy belong to security, while profile fields belong to the domain. M1 deliberately has no repository of its own and reuses M2's, avoiding two components writing the same table. **(2) Dashboard (M4) has no repository at all**: it is a read-model that composes counts published by M2 and M3. Had it queried their tables directly it would have coupled itself to their schemas, so any complaint-table change would silently break reporting. **(3) Search, filtering and the lifecycle live inside M3 rather than in separate modules**, because they operate on the same entity under the same authorisation rules; splitting them would have produced anaemic modules with high mutual coupling — the classic mistake of creating one module per screen. The single constraint enforced throughout is that **dependencies point downward only**: feature modules depend on infrastructure, never the reverse, and no cycles exist.

4.4 Module Description

Each module is specified below using the standard descriptor.

4.4.1 M1 — Authentication Module

Field	Description
Module Name	Authentication Module (M1)
Purpose	To establish user identity at registration and login, and to protect every other endpoint by verifying that identity and role on each request.
Responsibilities	Validate registration input; enforce email uniqueness; hash passwords with bcrypt; verify credentials without revealing which field was wrong; issue signed JWTs carrying user id and role; verify tokens and attach the caller's identity to the request; enforce role membership; seed the single administrator account at start-up.
Functional Requirements Covered	`REQ-1`, `REQ-2`, `REQ-3`, `REQ-4`, `REQ-5`, `REQ-6`, `REQ-7`, `REQ-8`; enables `REQ-20`, `REQ-27`
Inputs	Registration data (name, email, password, room number); login credentials; `Authorization: Bearer <token>` header on protected requests.
Outputs	`{ token, user }` on success; authenticated request context `{ userId, role }`; `401` / `403` / `409` errors.
Interfaces Provided	`POST /api/auth/register`, `POST /api/auth/login`; middleware `requireAuth()`, `requireRole(role)` consumed by all other modules.
Interfaces Required	User repository (M2) for lookup and creation; JWT and validation utilities, configuration and error types (M5).
Dependencies	M2, M5. No feature module depends on M1's internals — only on its middleware.
Internal Classes / Units	`AuthService` (register, login), `AuthController`, `AuthRoutes`, `AuthMiddleware` (requireAuth, requireRole), `JwtUtil`, `AdminSeeder`.

4.4.2 M2 — User Management Module

Field	Description
Module Name	User Management Module (M2)
Purpose	To own the persistent user record and expose safe operations for reading and modifying it.
Responsibilities	Persist and retrieve user rows; guarantee that password hashes never leave the module in a response; validate and apply profile changes; provide the administrator's list of registered students; expose the student count used by reporting.
Functional Requirements Covered	`REQ-38`, `REQ-39`, `REQ-40`, `REQ-41`; supports `REQ-1`–`REQ-4`, `REQ-34`

Inputs	Authenticated user id; profile fields (name, room number); lookup keys (id, email).
Outputs	Public user objects (id, name, email, room number, role, created date) — never the password hash; student collections; counts.
Interfaces Provided	`GET /api/users/me`, `PUT /api/users/me`, `GET /api/users` (administrator); repository operations `findByEmail`, `findById`, `createUser`, `countStudents` consumed by M1 and M4.
Interfaces Required	Authentication middleware (M1); database connection, validators, error types (M5).
Dependencies	M1 (guards), M5.
Internal Classes / Units	`UserService`, `UserController`, `UserRoutes`, `UserRepository` (sole owner of the `users` table).

4.4.3 M3 — Complaint Management Module

Field	Description
Module Name	Complaint Management Module (M3)
Purpose	To own the complaint entity throughout its life: creation by a student, controlled modification, administrator review and lifecycle advancement.
Responsibilities	Validate category and description; force the initial state to Pending regardless of client input; associate a complaint with its author; store and replace the optional image; enforce that only the owner may modify and only while Pending; permit reads by the owner or the administrator; provide paginated search and filtering joined with student details; apply status transitions, record remarks, maintain the last-updated time and record the resolution time exactly once.
Functional Requirements Covered	`REQ-9`–`REQ-32` (submission, tracking, administrator management, search and filter)
Inputs	Complaint data (category, description, optional image file); complaint identifier; search term, category/status filters, page and limit; status value and remarks.
Outputs	Complaint objects with status, timestamps, remarks and image reference; paginated result sets with student name and room; aggregate counts for the Dashboard module.
Interfaces Provided	`POST /api/complaints`, `GET /api/complaints/mine`, `GET /api/complaints/:id`, `PUT /api/complaints/:id`, `DELETE /api/complaints/:id`, `GET /api/complaints` (administrator), `PATCH /api/complaints/:id/status` (administrator); repository aggregate operations consumed by M4.
Interfaces Required	Authentication and role middleware (M1); upload middleware, constants, database connection, error types (M5); user data for joined listings (M2's table via the repository join).
Dependencies	M1, M2, M5.
Internal Classes / Units	`ComplaintService` (rules: `createComplaint`, `listMine`, `getOne`, `updateComplaint`, `deleteComplaint`, `listAll`, `updateStatus`), `ComplaintController`, `ComplaintRoutes`, `ComplaintRepository` (sole owner of the `complaints` table).

4.4.4 M4 — Dashboard & Analytics Module

Field	Description
Module Name	Dashboard & Analytics Module (M4)
Purpose	To provide role-appropriate summary statistics without duplicating the storage or rules owned by other modules.
Responsibilities	Compose a student's own complaint counts by state; compose system-wide totals, per-status counts, category distribution and a recent-complaints list; guarantee that every category and status key is present (zero-filled) so charts render completely.
Functional Requirements Covered	`REQ-33`, `REQ-34`, `REQ-35`, `REQ-36`, `REQ-37`

Inputs	Authenticated identity and role (no user-supplied parameters).
Outputs	Student view: total, pending, in-progress, resolved, closed. Administrator view: total students, total complaints, counts by status, counts by category, five most recent complaints.
Interfaces Provided	`GET /api/dashboard/student`, `GET /api/dashboard/admin`.
Interfaces Required	Aggregate operations of M3's repository; student count from M2; role middleware (M1); constants (M5).
Dependencies	M1, M2, M3, M5. Nothing depends on M4 — it is a leaf module, so it can be modified or removed without affecting the transactional core.
Internal Classes / Units	`DashboardService`, `DashboardController`, `DashboardRoutes`. Deliberately no repository.

4.4.5 M5 — Infrastructure & Cross-Cutting Layer

Field	Description
Module Name	Infrastructure & Cross-Cutting Layer (M5)
Purpose	To provide the shared technical services every feature module needs, so that cross-cutting behaviour is defined once and applied uniformly.
Responsibilities	Load and validate configuration and refuse to start with insecure production settings; open the database connection and apply the schema; apply security headers and rate limiting; accept and filter uploaded images; translate any thrown error into a uniform JSON response with the correct status; log requests; provide token and validation helpers; hold the single definition of categories, statuses and roles.
Functional Requirements Covered	None directly; realises **NFR-5**, **NFR-8**–**NFR-13** and supports **REQ-10**, **REQ-12**, **REQ-28**
Inputs	Environment variables; raw HTTP requests; uploaded files; thrown errors.
Outputs	Typed configuration object; database handle; stored image files; uniform error responses; log lines.
Interfaces Provided	`config`, `constants`, `db`, `securityHeaders`, `authLimiter`, `uploadImage`, `errorHandler`, `AppError`, `requestLogger`, `signToken` / `verifyToken`, validators.
Interfaces Required	Node.js runtime, filesystem, database engine.
Dependencies	None within the application (lowest layer).
Internal Classes / Units	`Config`, `Constants`, `Database`, `SecurityMiddleware`, `UploadMiddleware`, `ErrorHandler`, `AppError`, `RequestLogger`, `JwtUtil`, `Validators`, `Seeder`.

4.5 Module Interaction

Figure 4.5 shows the runtime collaboration between modules for the system's principal operations. Arrows are labelled with the interaction that occurs; the numbering follows the order in which a typical complaint progresses through the system.

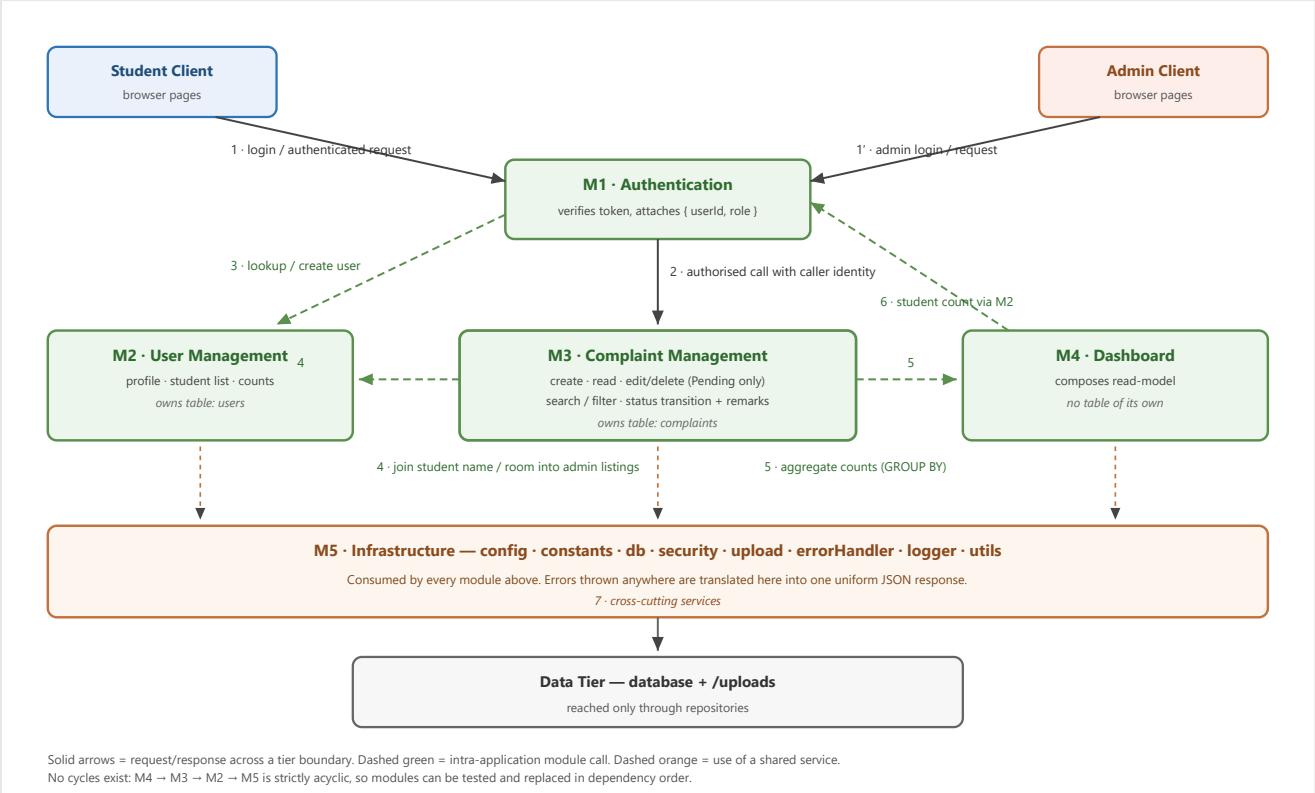

Figure 4.5 — Module Interaction Diagram.

> **Design Rationale.** Interactions were kept **acyclic and one-directional** (M4 → M3 → M2 → M5) so that each module can be understood, tested and replaced without reasoning about feedback loops. The most important decision visible here is that **M4 calls M3 and M2 rather than the database**: reporting is a consumer of the domain, not a peer of it, so if the complaint schema changes only M3's repository must adapt. The second is that **M1 sits in front of every other module** as middleware rather than being called by them — authentication is therefore impossible to forget when adding a new endpoint, because a route without a guard simply has no access to a caller identity. A trade-off accepted here is that M3's administrator listing performs a SQL join onto the `users` table owned by M2. A stricter design would have M3 request user details through M2's service, but that would replace one indexed join with N follow-up calls; the join was chosen for performance, and the coupling is confined to a single query in one repository.

4.6 Package Diagram

The physical source organisation mirrors the logical decomposition, so a developer can move from a module in Figure 4.1 to the directory that implements it without translation.

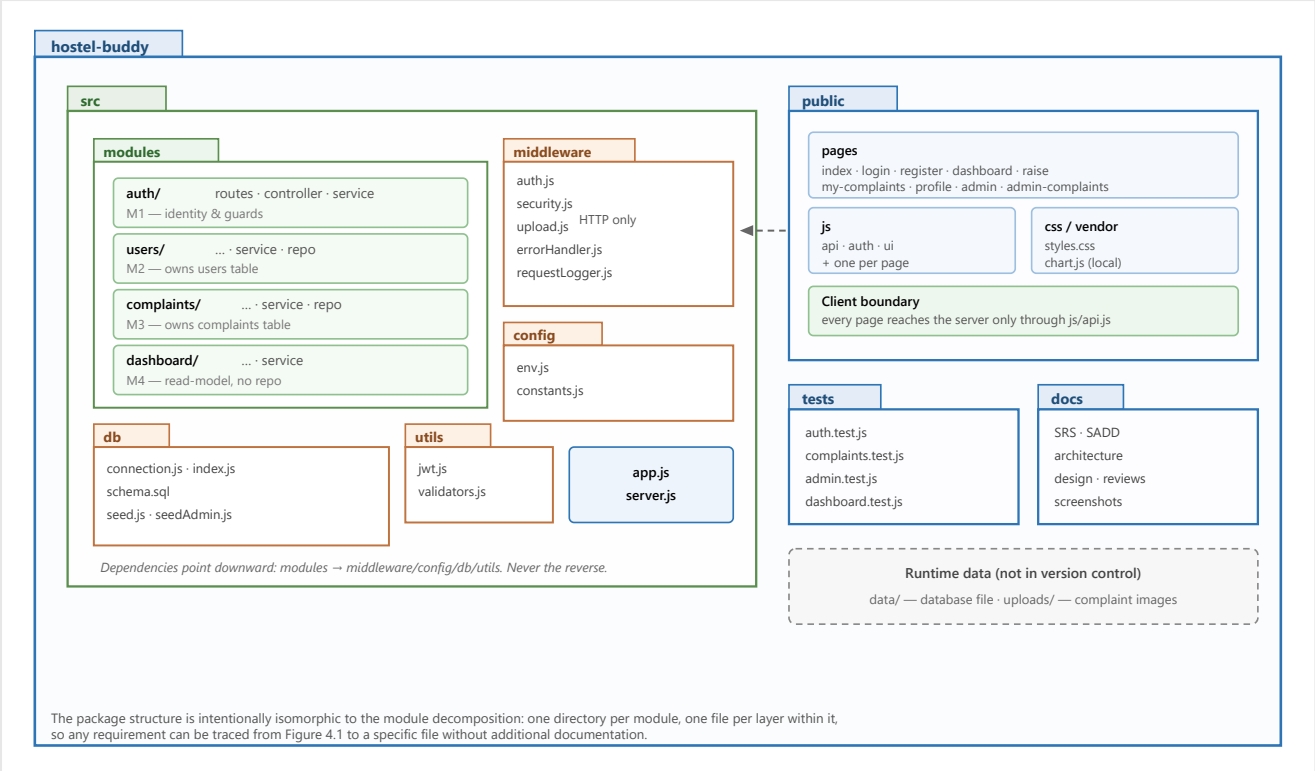

Figure 4.6 — UML Package Diagram (source organisation).

Design Rationale. The package layout is organised **by feature first and by layer second** (`modules/complaints/…service`) rather than the reverse (`services/complaintService`). Layer-first packaging scatters a single change across four distant directories; feature-first keeps everything a developer must edit for one requirement in one folder, which measurably reduces navigation cost and merge conflicts on a four-person team. Shared infrastructure sits outside `modules/` precisely because it belongs to no feature, and the downward-only dependency rule is visible structurally: nothing inside `middleware/` , `config/` , `db/` or `utils/` imports from `modules/` . Keeping `public/` a sibling of `src/` , connected only by HTTP, makes the client's replaceability explicit — it shares no code with the server and therefore cannot accidentally depend on server internals. Finally, runtime directories are excluded from version control so that a clone contains only source, and the database is rebuilt deterministically by the schema and seed scripts.

5. Interface Design

5.1 User Interface Overview

The interface comprises nine pages, partitioned by role. All pages share one stylesheet, one navigation component and one network wrapper, so behaviour is consistent and cross-cutting concerns (token handling, session expiry, error display) are implemented once.

#	Screen	Access	Purpose and principal controls
1	Home / Landing	Public	Introduces the system; routes to Login or Register. Redirects automatically to the correct dashboard if a session already exists.
2	Login	Public	Email and password; single form serving **both roles** — the server determines the role from the credentials and the client routes accordingly.
3	Register	Public	Name, email, room number, password; creates a student account only.
4	Student Dashboard	Student	Five summary cards (Total, Pending, In Progress, Resolved, Closed) and shortcuts to raise or review complaints.
5	Raise Complaint	Student	Category selector, description with live character counter, optional image picker, submit.
6	My Complaints	Student	Table of the student's complaints with status badges and dates; View for any complaint; **Edit and Delete rendered only for Pending rows**; detail and edit presented in a modal.
7	Profile	Student	View and update name and room number; email shown read-only.
8	Admin Dashboard	Administrator	Six statistic cards, a category distribution doughnut chart, a status bar chart and the five most recent complaints.
9	All Complaints	Administrator	Search box, category and status filters, paginated table, and a Manage modal for changing status and recording remarks.

Two conventions carry meaning across the interface: a **status badge** uses one fixed colour per lifecycle state (amber Pending, blue In Progress, green Resolved, grey Closed) on every screen; and **destructive or role-specific controls are not rendered at all** when unavailable, rather than being shown disabled, which prevents users from attempting actions the server will reject.

5.2 Module Interfaces

The table specifies the programmatic interface each module publishes. These are the only supported entry points; nothing outside a module may reach past them.

Interface	Input	Output	Description
M1 — Authentication			
`POST /api/auth/register`	name, email, password, room_number	`201` { token, user }	Creates a student account; rejects duplicates with `409` and invalid input with `400`.
`POST /api/auth/login`	email, password	`200` { token, user }	Verifies credentials for either role; returns a single generic `401` on any failure.

`requireAuth()`	Authorization header	request context { userId, role }	Middleware; rejects missing/invalid/expired tokens with `401`.
`requireRole(role)`	request context	pass or `403`	Middleware; restricts an endpoint to one role.
M2 — User Management			
`GET /api/users/me`	authenticated identity	`200` user object	Returns the caller's own profile without the password hash.
`PUT /api/users/me`	name, room_number	`200` updated user	Validates and applies profile changes; email and role are immutable.
`GET /api/users`	administrator identity	`200` user array	Lists registered students (administrator excluded).
`UserRepository.findByEmail / findById / createUser / countStudents`	email · id · user data · —	user row · user · created user · integer	Internal repository operations consumed by M1 and M4.
M3 — Complaint Management			
`POST /api/complaints`	category, description, image (multipart)	`201` complaint	Creates a complaint forced to **Pending** and owned by the caller.
`GET /api/complaints/mine`	student identity	`200` complaint array	Returns the caller's complaints, newest first.
`GET /api/complaints/:id`	complaint id	`200` complaint	Readable by the owner or the administrator; otherwise `403`.
`PUT /api/complaints/:id`	category, description, image	`200` complaint	Owner-only and Pending-only; otherwise `403` / `409`.
`DELETE /api/complaints/:id`	complaint id	`200` { deleted, id }	Owner-only and Pending-only; removes the stored image as well.
`GET /api/complaints`	q, category, status, page, limit	`200` { data[], pagination }	Administrator search and filter, joined with student name and room.
`PATCH /api/complaints/:id/status`	status, admin_remarks	`200` complaint	Administrator lifecycle transition; sets the resolution time on first Resolved only.
M4 — Dashboard & Analytics			
`GET /api/dashboard/student`	student identity	`200` counts by state	Personal totals for the five summary cards.
`GET /api/dashboard/admin`	administrator identity	`200` statistics object	Totals, per-status and per-category counts (zero-filled), and recent complaints.
M5 — Infrastructure			
`uploadImage`	multipart request	stored file reference	Accepts PNG/JPEG/WEBP within the size limit; rejects others with `400`.
`errorHandler` / `AppError`	thrown error	uniform JSON error	Maps any failure to `{ error: { message, code } }` with the correct status.
`config` / `constants`	environment	typed settings, enums	Single source of truth for configuration and controlled vocabularies.

5.3 External Interfaces

Version 1.0 consumes **no external service APIs**. The externally visible interface is the system's own REST API, described above and specified fully in Appendix B. Its characteristics:

Aspect	Specification
Style	Resource-oriented REST over HTTP; nouns as paths, verbs as methods (`GET` , `POST` , `PUT` , `PATCH` , `DELETE`).
Base path	`/api` ; resources `/auth` , `/users` , `/complaints` , `/dashboard` .
Request format	`application/json` , except complaint create/update which use `multipart/form-data` to carry the image.
Response format	`application/json` ; a uniform error envelope `{ error: { message, code } }` for every failure.
Authentication	`Authorization: Bearer <JWT>` on all protected endpoints.
Status codes	`200` / `201` success; `400` validation; `401` unauthenticated; `403` forbidden; `404` not found; `409` conflict (duplicate email, non-Pending modification); `429` rate limited; `500` internal.
Static resources	Frontend assets served from the application root; uploaded images served read-only from `/uploads` .
Future integrations	An SMTP gateway and a push-notification provider attach at a defined hook in the complaint service after a successful status transition (§13.3), requiring no change to the API contract.

5.4 Communication Interfaces

Concern	Design
Transport protocol	HTTP/1.1. In deployment, **HTTPS is mandatory** (`NFR-12`), terminated by a reverse proxy in front of the application; HSTS is emitted so browsers refuse subsequent plaintext connections.
Network stack	TCP/IP; the application listens on a configurable port (default 4000).
Data encoding	UTF-8 JSON for structured data; `multipart/form-data` for image upload; binary image bytes on retrieval.
Session propagation	Stateless — the signed JWT travels in the `Authorization` header on each request. No server-side session and no session cookie, so no CSRF token exchange is required.
Origin policy	Client and API are served from a single origin, so no cross-origin configuration is needed. A future separately-hosted client would require an explicit CORS allow-list rather than a wildcard.
Security headers	Content Security Policy restricting scripts to the application's own origin, plus `X-Content-Type-Options` , `X-Frame-Options` , `Referrer-Policy` and HSTS.
Rate limiting	Authentication endpoints are limited per client address in production to mitigate brute-force attempts; exceeding the limit returns `429` .
Payload limits	Image uploads capped at 5 MB and restricted by MIME type; administrator listings are paginated with a bounded maximum page size to keep responses small.
Failure semantics	All operations are idempotent except complaint creation. A client receiving `401` clears its stored token and returns the user to login.

6. Database Design

6.1 Database Design Overview

Hostel Buddy uses a **relational database**. The decision follows directly from the shape of the data described in the SRS.

Why relational rather than NoSQL

- **The schema is known, small and stable.** Two entities with fixed attributes were fully specified before implementation. The schema flexibility a document store offers would buy nothing here.
- **The data is genuinely relational.** Every complaint belongs to exactly one student, and the administrator's principal screen requires complaint records joined with the author's name and room (`REQ-29`). A join is the natural expression of this; duplicating student details into each complaint document would create an update anomaly the moment a student changes rooms.
- **Controlled vocabularies must be enforced.** Categories and statuses are closed sets (`REQ-10`, `REQ-28`). `CHECK` constraints let the database itself reject an invalid value, giving a guarantee that survives any application bug.
- **Reporting is aggregate-heavy.** `REQ-34`–`REQ-36` are naturally expressed as `COUNT … GROUP BY`, which a relational engine executes efficiently against an index.
- **Integrity matters more than write throughput.** `NFR-5` requires that no complaint be lost; ACID transactions and foreign keys provide this directly, whereas eventual consistency would be an unnecessary risk at this scale.

The concrete engine is **SQLite** accessed through the driver built into the Node.js runtime, running in write-ahead-logging mode. It stores the entire database in a single file, requires no separate server process and needs no native compilation. The schema is written in portable SQL so that migrating to PostgreSQL for a multi-instance deployment affects only the repository layer (§11.1).

6.2 ER Diagram

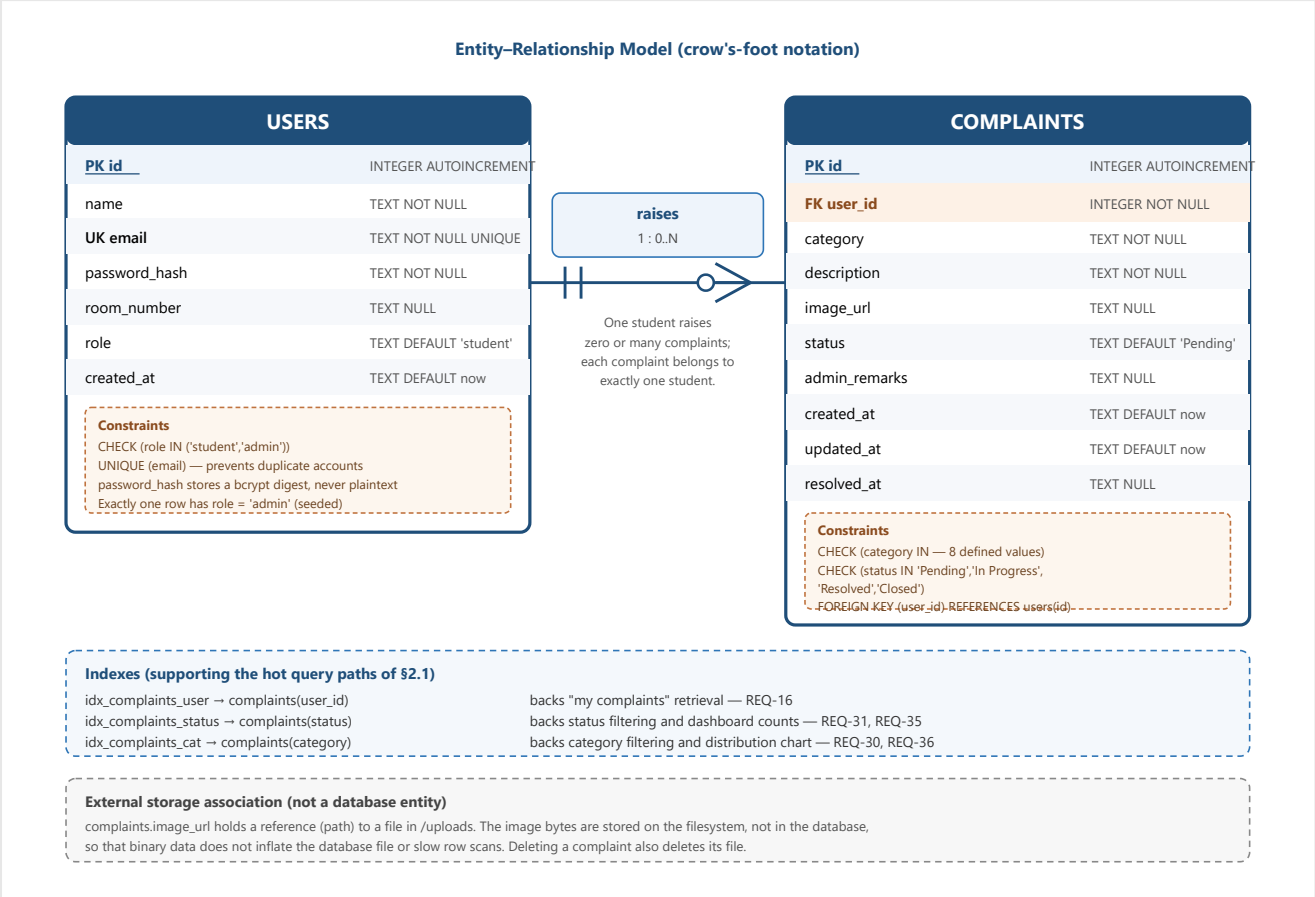

Figure 6.2 — ER Diagram for Hostel Buddy.

> **Design Rationale.** The model is deliberately **two entities, not more**. Three plausible extra tables were considered and rejected. A separate *Category* table would allow categories to be added without a schema change, but v1.0 fixes the eight categories by requirement (**REQ-10**), and a `CHECK` constraint enforces the same closed set with one fewer join on the hottest query — a lookup table becomes worthwhile only when categories become user-manageable. A separate *Status History* table would provide a full audit trail of transitions, but the SRS requires only the *current* status plus the submission and resolution dates (**REQ-17**, **REQ-25**, **REQ-26**), so the three timestamp columns satisfy the requirement without the cost of a growing history table; this is recorded as the main extension point should auditing become a requirement. A separate *Admin* table was rejected because an administrator differs from a student only by role: one table with a `role` column keeps authentication uniform and avoids duplicating credential logic. Finally, the choice to store **image paths rather than image bytes** keeps rows narrow so that the frequent list and aggregate queries stay fast, at the accepted cost that database and filesystem must be backed up together.

6.3 Entity Description

USERS

Represents every person who can authenticate, whether student or administrator. A row is created when a visitor registers (students) or when the application starts for the first time (the single administrator). The `role` column is the sole discriminator and drives all authorisation decisions. The password is stored only as a bcrypt digest and is never returned by any interface. `room_number` is optional at the data level because

a student may register before being allotted a room, and it is nullable for the administrator, who has no room. Deletion is not part of the normal lifecycle: complaints reference users, so removing a user would orphan their complaint history.

COMPLAINTS

Represents a single maintenance issue reported by one student. It is created in the **Pending** state and is thereafter advanced only by the administrator. The record carries three distinct kinds of information: the *description of the problem* (category, description, optional image), the *workflow state* (status, admin remarks), and the *chronology* (created, last updated, resolved). Separating these is what allows the student view (`REQ-17`) to show submission date, current status, remarks and resolution date from a single row. `resolved_at` is written exactly once — the first time the complaint reaches Resolved — so that a later transition to Closed, or an edit of remarks, cannot falsify the resolution time.

Relationship: *raises* (USERS 1 : 0..N COMPLAINTS)

A student may have no complaints at all (a newly registered account) or many. Every complaint must have exactly one author, enforced by a `NOT NULL` foreign key. This single relationship is what makes ownership checks possible: comparing `complaints.user_id` with the authenticated caller's id is the mechanism behind `REQ-20`. The administrator participates in the relationship only as a reader and never as an author, which is why no second relationship to USERS is modelled.

6.4 Data Dictionary

Entity	Attribute	Type	Key / Nullability	Description
USERS	`id`	INTEGER	PK, auto-increment	Surrogate primary key identifying the user.
	`name`	TEXT (≤100)	NOT NULL	Full name shown in the interface and in administrator listings.
	`email`	TEXT	UNIQUE, NOT NULL	Login identifier; normalised to lower case; uniqueness prevents duplicate accounts (`REQ-2`).
	`password_hash`	TEXT	NOT NULL	bcrypt digest (cost 10) including its salt. Never returned by any API (`REQ-3`).
	`room_number`	TEXT (≤20)	NULL	Hostel room identifier, e.g. "B-204"; searchable by the administrator (`REQ-29`).
	`role`	TEXT	NOT NULL, default `student`	Authorisation discriminator; constrained to `student` or `admin`.
	`created_at`	TEXT (ISO datetime)	NOT NULL, default now	Account creation timestamp; orders the student list.
COMPLAINTS	`id`	INTEGER	PK, auto-increment	Complaint identifier; shown to users as the searchable Complaint ID (`REQ-29`).
	`user_id`	INTEGER	FK → users(id), NOT NULL	Author of the complaint; the basis of every ownership check.
	`category`	TEXT	NOT NULL, CHECK	One of: Electricity, Plumbing, Water Supply, Wi-Fi, Cleaning, Furniture, Security, Other (`REQ-10`).

COMPLAINTS	`description`	TEXT (≤1000)	NOT NULL	Free-text account of the issue (**REQ-11**).
	`image_url`	TEXT	NULL	Path to the optional uploaded image under `/uploads` (**REQ-12**).
	`status`	TEXT	NOT NULL, default `Pending`, CHECK	Current lifecycle state: Pending, In Progress, Resolved or Closed (**REQ-13**, **REQ-28**).
	`admin_remarks`	TEXT (≤1000)	NULL	Administrator's note to the student, visible in the tracking view (**REQ-24**).
	`created_at`	TEXT (ISO datetime)	NOT NULL, default now	Submission timestamp; default sort key, newest first (**REQ-13**).
	`updated_at`	TEXT (ISO datetime)	NOT NULL, default now	Time of the most recent change of any kind (**REQ-25**).
	`resolved_at`	TEXT (ISO datetime)	NULL	Set exactly once, when the complaint first becomes Resolved; never reset (**REQ-26**).

6.5 Database Constraints

Constraint type	Definition	Purpose and requirement served
Primary key	`users.id`, `complaints.id`	Guarantees unique, stable identity for every row. Surrogate integer keys are used rather than natural keys so that a user changing email does not break existing references.
Foreign key	`complaints.user_id → users.id`	Guarantees every complaint has a real author; prevents orphan records. Enforcement is enabled explicitly on each connection.
Unique	`users.email`	Prevents duplicate accounts (**REQ-2**). Acts as the authoritative backstop: even if two registrations race past the application's existence check, the database rejects the second, which the service converts into a clear `409` response.
Check (role)	`role IN ('student','admin')`	Makes an unknown role impossible to store, so authorisation logic never encounters an unexpected value.
Check (category)	8 permitted category values	Enforces the controlled vocabulary of **REQ-10** at the storage level, keeping the category distribution chart meaningful.
Check (status)	4 permitted lifecycle states	Enforces **REQ-28**; guarantees no complaint can exist outside the defined workflow.
Not null	name, email, password_hash, role, user_id, category, description, status, created_at, updated_at	Encodes the mandatory fields of the requirements so that incomplete records cannot be persisted.
Default values	`status='Pending'`, `role='student'`, timestamps = current time	Makes the safe value the automatic one: a complaint inserted without a status is Pending, and a user created without a role is a student, so a privilege cannot be granted by omission.
Application-level rules	Length limits (name 100, room 20, description 1000, remarks 1000); email format; password ≥ 6 characters; image type and 5 MB size	Validated in the service layer where a helpful message can be produced, in addition to the structural constraints above (**REQ-8**, **REQ-12**, **REQ-39**).

Defence in depth. Enumerations and uniqueness are enforced twice — once in the service layer, where a friendly error message can be returned, and once in the schema, where correctness is guaranteed regardless of the calling code. The application layer exists for usability; the database layer exists for truth.

6.6 Normalization

The schema is in **Third Normal Form (3NF)**. The progression is shown below.

Form	Requirement	Evidence in this schema
1NF	Atomic attributes; no repeating groups.	Every column holds a single scalar value. A complaint carries exactly one category and one image reference rather than a list, so no multi-valued attribute exists.
2NF	1NF and no partial dependency on part of a composite key.	Both tables use a single-column surrogate primary key, so no composite key exists and partial dependency is impossible by construction.
3NF	2NF and no transitive dependency of a non-key attribute on another non-key attribute.	Every non-key column depends only on its table's key. Critically, the student's `name` and `room_number` are stored **only** in USERS — the administrator's listing obtains them by joining rather than by duplicating them into COMPLAINTS, which would have created a transitive dependency and an update anomaly whenever a student changed rooms.

Deliberate design decisions regarding normalisation

- **Categories and statuses are stored as constrained text, not foreign keys to lookup tables.** Strictly, a lookup table would push the design further toward a fully normalised catalogue. It was rejected because the vocabularies are fixed by requirement, `CHECK` constraints provide the same integrity guarantee, and avoiding two extra joins keeps the most frequent queries — the administrator listing and the dashboard aggregation — simpler and faster. Should categories become administrator-manageable, promoting them to a table is a localised change affecting one repository.
- `resolved_at` **is retained despite being derivable in principle from a status history.** Because no history table exists, this column is not redundant — it is the only record of when resolution occurred, and storing it avoids an expensive reconstruction on every read.
- **No denormalisation for performance was necessary.** Measured query times over a realistic dataset remained at or below approximately one millisecond, so the schema retains full 3NF integrity with no cached counts or duplicated columns.

7. Detailed Design

7.1 Object-Oriented Design Overview

Each module of §4 is realised as a small collection of collaborating classes, one per architectural layer. The design applies object-oriented principles as follows.

Principle	Application
Abstraction	Each class exposes the operations meaningful to its collaborators and hides how they are achieved. `ComplaintRepository` publishes `findByUser`, `search` and `updateStatus`; callers never see SQL, table names or pagination arithmetic. Similarly `JwtUtil` exposes only `signToken` / `verifyToken`, hiding the algorithm and expiry policy.
Encapsulation	State and the rules governing it are kept together. The complaint lifecycle is manipulated exclusively through `ComplaintService.updateStatus()`; no other class may write the `status` or `resolved_at` fields. `UserRepository` encapsulates the users table and deliberately exposes two distinct read operations — one returning the full row for credential verification, one returning a public projection without the password hash — so that leaking the digest requires actively choosing the wrong method.
Separation of interface and implementation	Services depend on repository *operations*, not on the database engine. Replacing SQLite with PostgreSQL changes repository internals while services, controllers and routes remain untouched.
Polymorphism	Used where it earns its place rather than for its own sake. Express middleware are interchangeable objects implementing the same `(request, response, next)` contract, so guards, upload handling and the error handler compose freely into a pipeline. `AppError` extends the built-in `Error` type, so a single `catch` handles both deliberate application failures and unexpected runtime errors, treating each according to its actual type.
Inheritance	Applied sparingly — only `AppError extends Error`. Deep hierarchies were avoided because the domain has one entity type with no natural subtypes; composition and explicit delegation were preferred throughout, in keeping with the low-coupling goal.
Single responsibility	Controllers translate HTTP, services decide, repositories persist. A class that begins to do two of these is a signal to split it.
Dependency direction	Controllers depend on services; services depend on repositories; repositories depend on the database connection. No reverse dependency exists, so the layers can be unit-tested in isolation from the top down.

7.2 Class Diagram

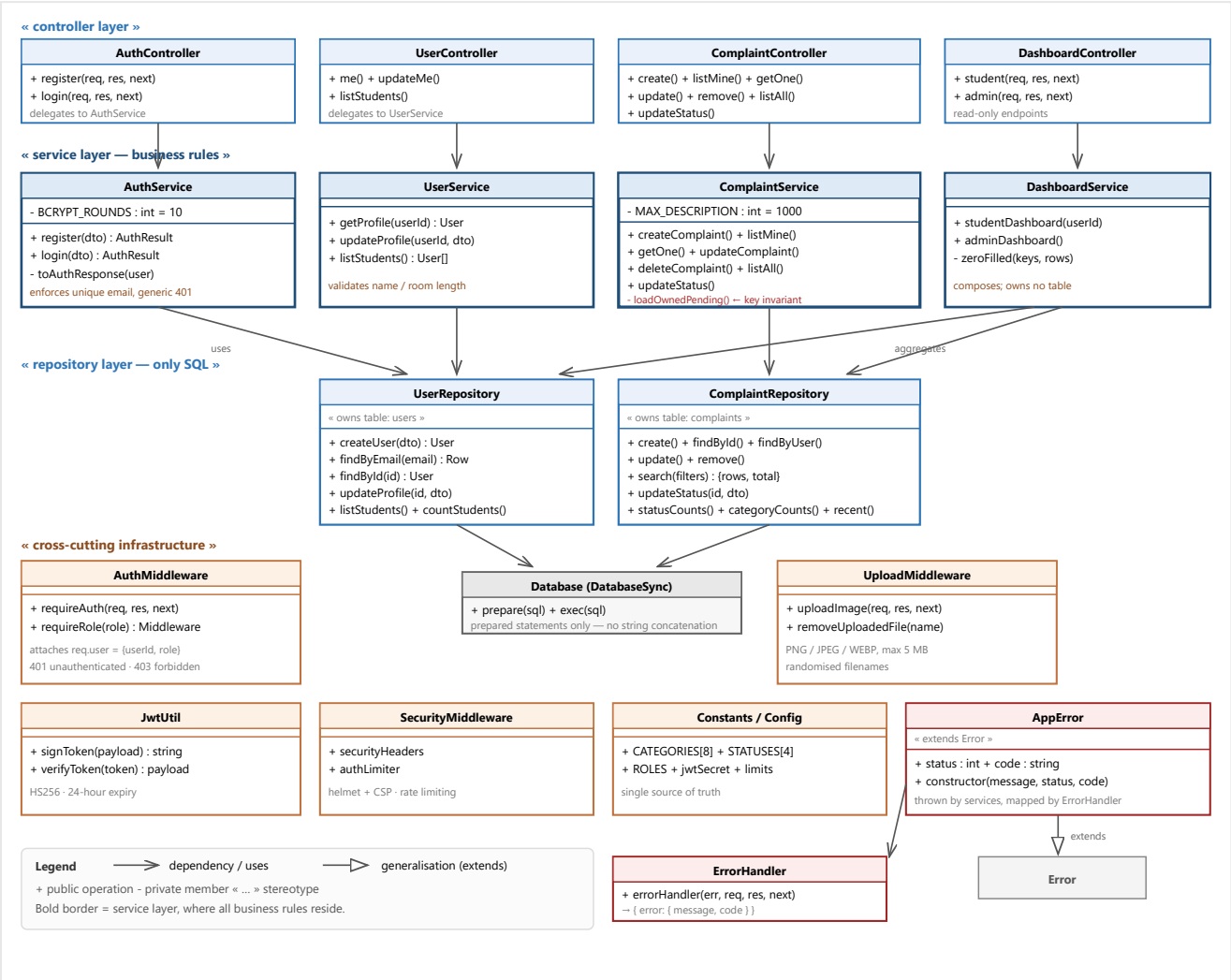

Figure 7.1 — Class Diagram (principal classes and their relationships).

> **Design Rationale.** The class structure repeats one shape — controller, service, repository — for every module, which was chosen over a richer domain model for a specific reason: the domain has **one entity with a state machine and a handful of invariants**, not a web of interacting objects. A full domain-model approach (a `Complaint` class owning its own transition methods and persistence) would add mapping machinery without removing any real complexity at this size. The most significant single element in the diagram is the private helper `ComplaintService.loadOwnedPending()` : the ownership and Pending-only invariants of **REQ-18**–**REQ-20** are implemented *once* there and reused by both edit and delete, so the two operations cannot drift apart or be individually forgotten. `AuthService` deliberately holds no repository of its own and borrows `UserRepository` , keeping a single writer for the users table. `DashboardService` deliberately holds no repository at all. Inheritance appears exactly once (`AppError extends Error`) because that is the only place where substitutability is genuinely useful — a single catch site can then handle deliberate and accidental failures correctly by type.

7.3 Class Description

Class	Layer	Responsibility and key design notes

`AuthService`	Service (M1)	Registers students and authenticates users. Validates input, normalises the email, enforces uniqueness, hashes the password with bcrypt and issues a token. Returns an identical generic error for "unknown email" and "wrong password" so that the API cannot be used to enumerate registered accounts. Catches the database uniqueness violation as a backstop and converts it into a clear conflict response.
`UserService`	Service (M2)	Reads and updates profiles. Permits only `name` and `room_number` to change; email and role are immutable by construction, so privilege escalation through a profile update is impossible.
`ComplaintService`	Service (M3)	The heart of the system. Validates category against the controlled vocabulary and enforces the description limit; forces the initial status to Pending, ignoring any client-supplied value; resolves ownership for reads (owner or administrator); enforces the Pending-only rule for edit and delete through `loadOwnedPending()` ; applies status transitions and sets `resolved_at` only on the first transition into Resolved.
`DashboardService`	Service (M4)	Builds the two read-models by composing repository aggregates. `zeroFilled()` guarantees every status and category key is present even when its count is zero, so the client can render a complete chart without defensive logic.
`UserRepository`	Repository (M2)	Sole owner of the `users` table. Exposes `findByEmail` returning the complete row (including the hash) strictly for credential verification, and `findById` / `listStudents` returning a public projection that omits the hash — making the safe path the default one.
`ComplaintRepository`	Repository (M3)	Sole owner of the `complaints` table. Provides CRUD, the joined and paginated `search()` used by the administrator, and the aggregate operations consumed by the dashboard. Builds filter clauses with bound parameters exclusively, so search input cannot alter query structure.
`AuthMiddleware`	Infrastructure	Verifies the bearer token and attaches the caller's identity to the request; `requireRole()` is a factory returning a guard for a specific role. Because identity arrives only through this component, an endpoint that omits it simply has no caller to act upon.
`UploadMiddleware`	Infrastructure	Accepts a single optional image, restricts type and size, stores it under a randomised filename, and translates upload failures into the standard error shape. Also removes orphaned files when a later validation step rejects the request.
`AppError` / `ErrorHandler`	Infrastructure	`AppError` carries an HTTP status and a machine-readable code alongside the message. `ErrorHandler` is the single place that converts any failure into the uniform response, logging server faults while returning a safe generic message for them.
`JwtUtil` , `Constants` , `Config`	Infrastructure	Isolate token mechanics, controlled vocabularies and environment configuration. Adding a complaint category is a one-line change in `Constants` plus the corresponding schema constraint.

7.4 Sequence Diagrams

Four scenarios are modelled: the two most frequent operations, the security-critical negative path, and the administrator workflow that drives the complaint lifecycle.

7.4.1 Student registration and login (UC-1, UC-2)

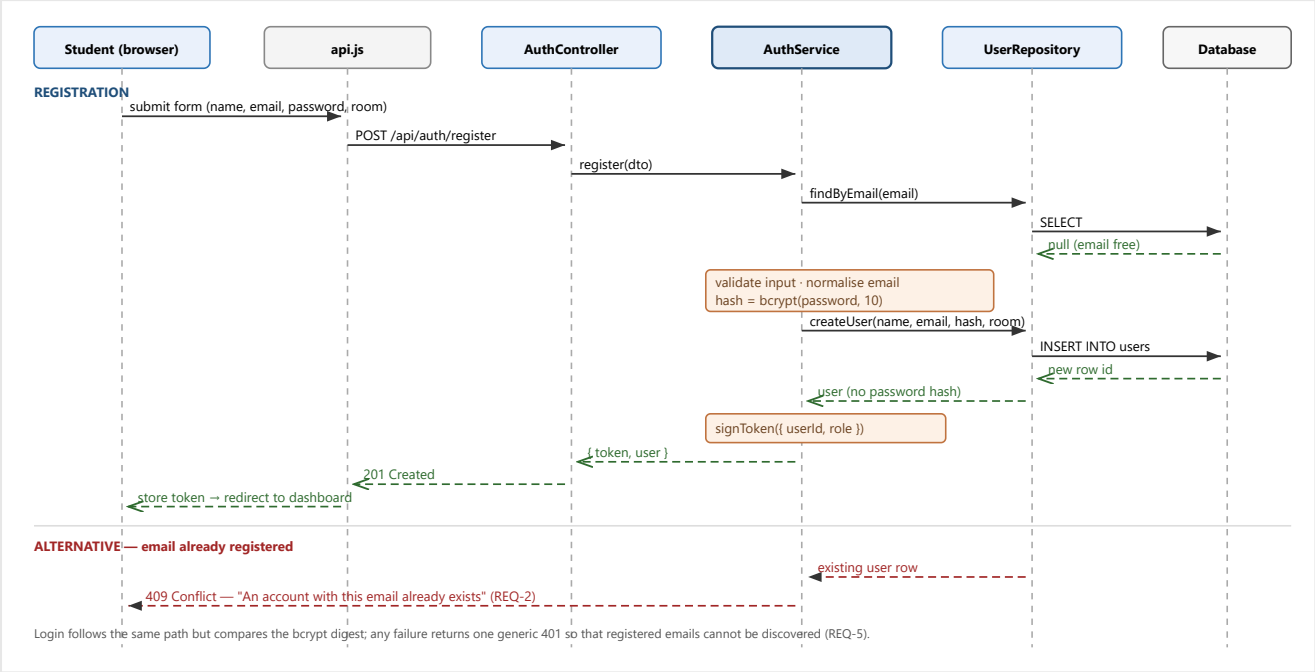

Figure 7.2 — Sequence: student registration, with the duplicate-email alternative path.

7.4.2 Raising a complaint with an image (UC-4, UC-5)

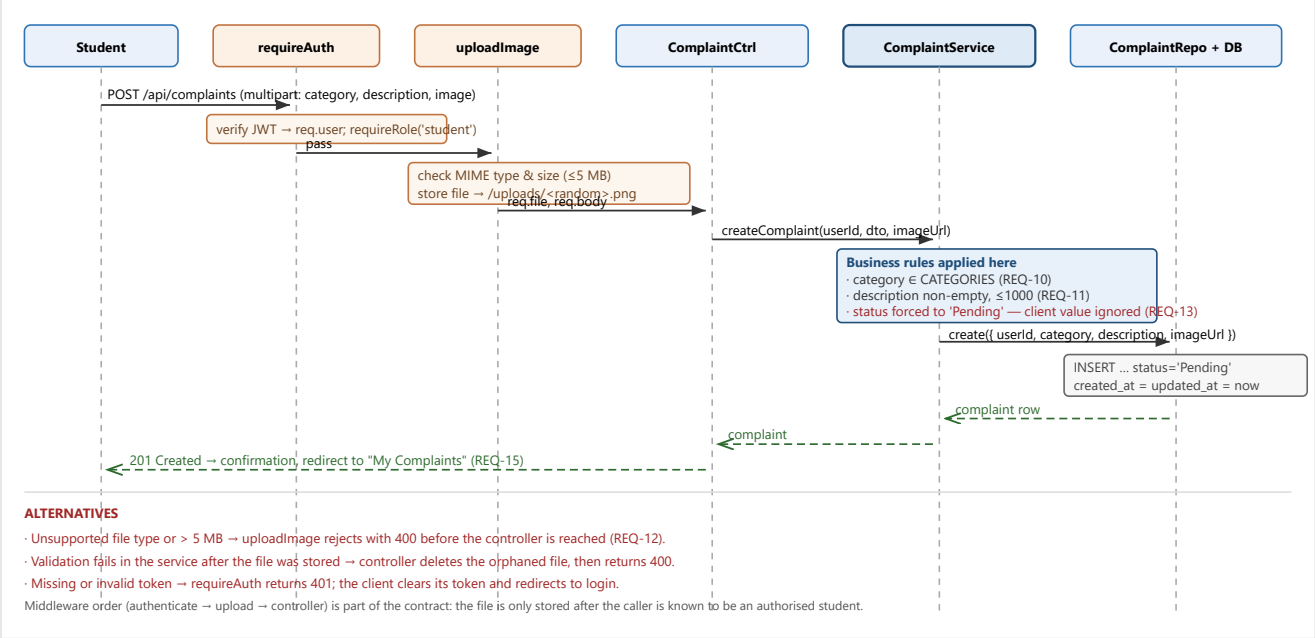

Figure 7.3 — Sequence: raising a complaint with an optional image, including failure paths.

7.4.3 Administrator resolves a complaint (UC-11, UC-12)

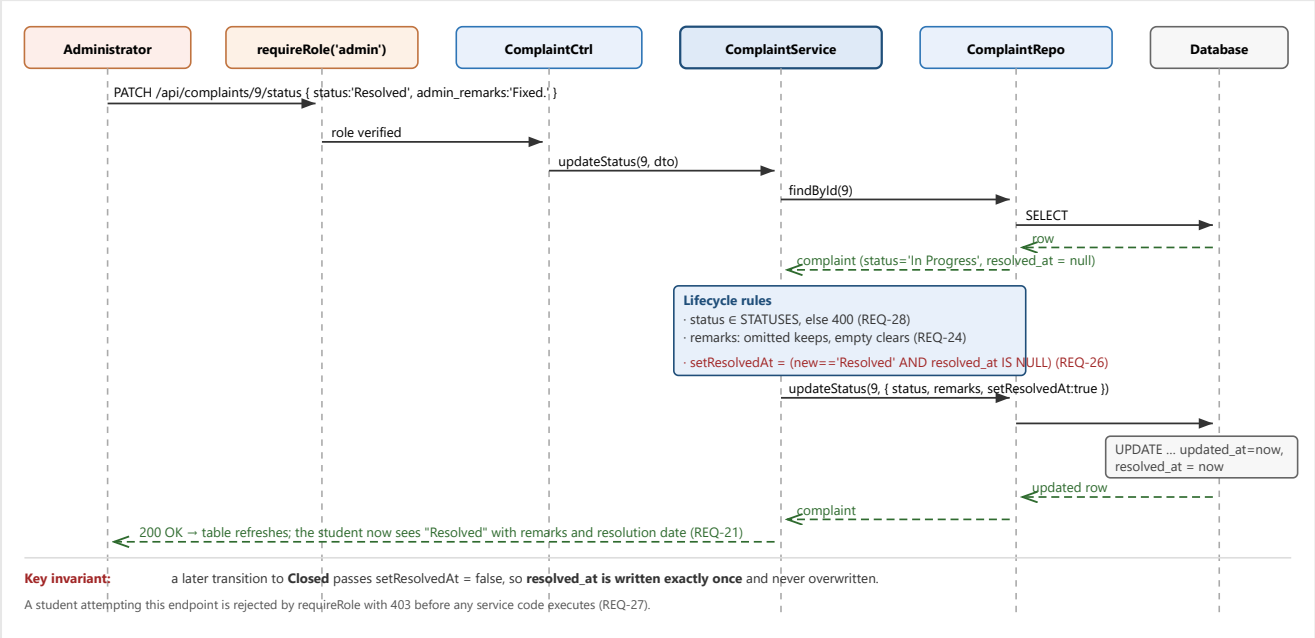

Figure 7.4 — Sequence: administrator advancing a complaint to Resolved.

7.4.4 Student attempts to edit a non-Pending complaint — negative path (UC-6)

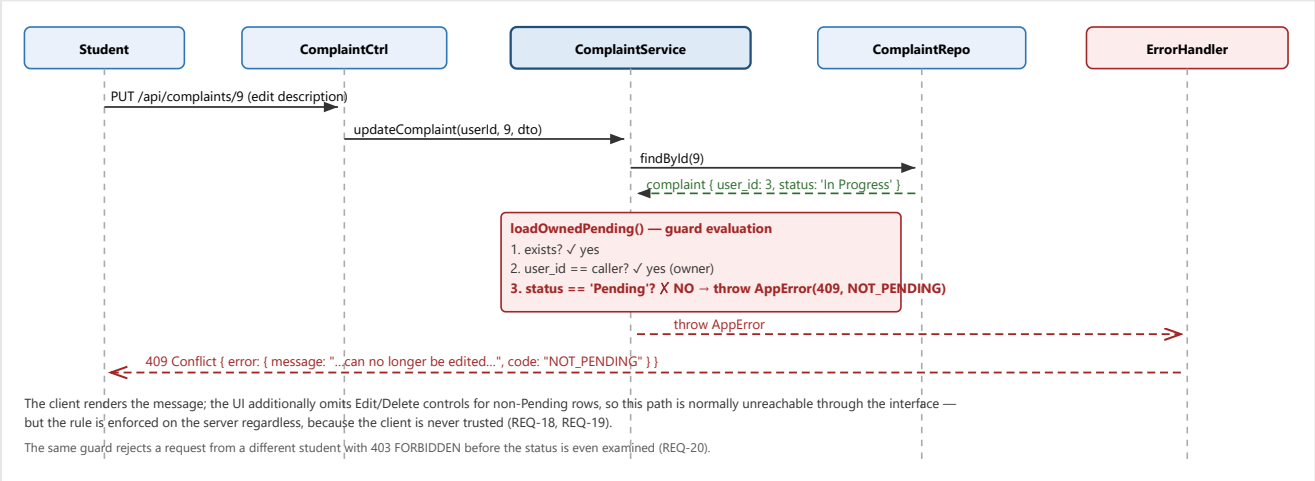

Figure 7.5 — Sequence: rejected edit of a non-Pending complaint (security-critical negative path).

7.5 Activity Diagram

The diagram traces the complete complaint workflow across both actors, from submission to closure.

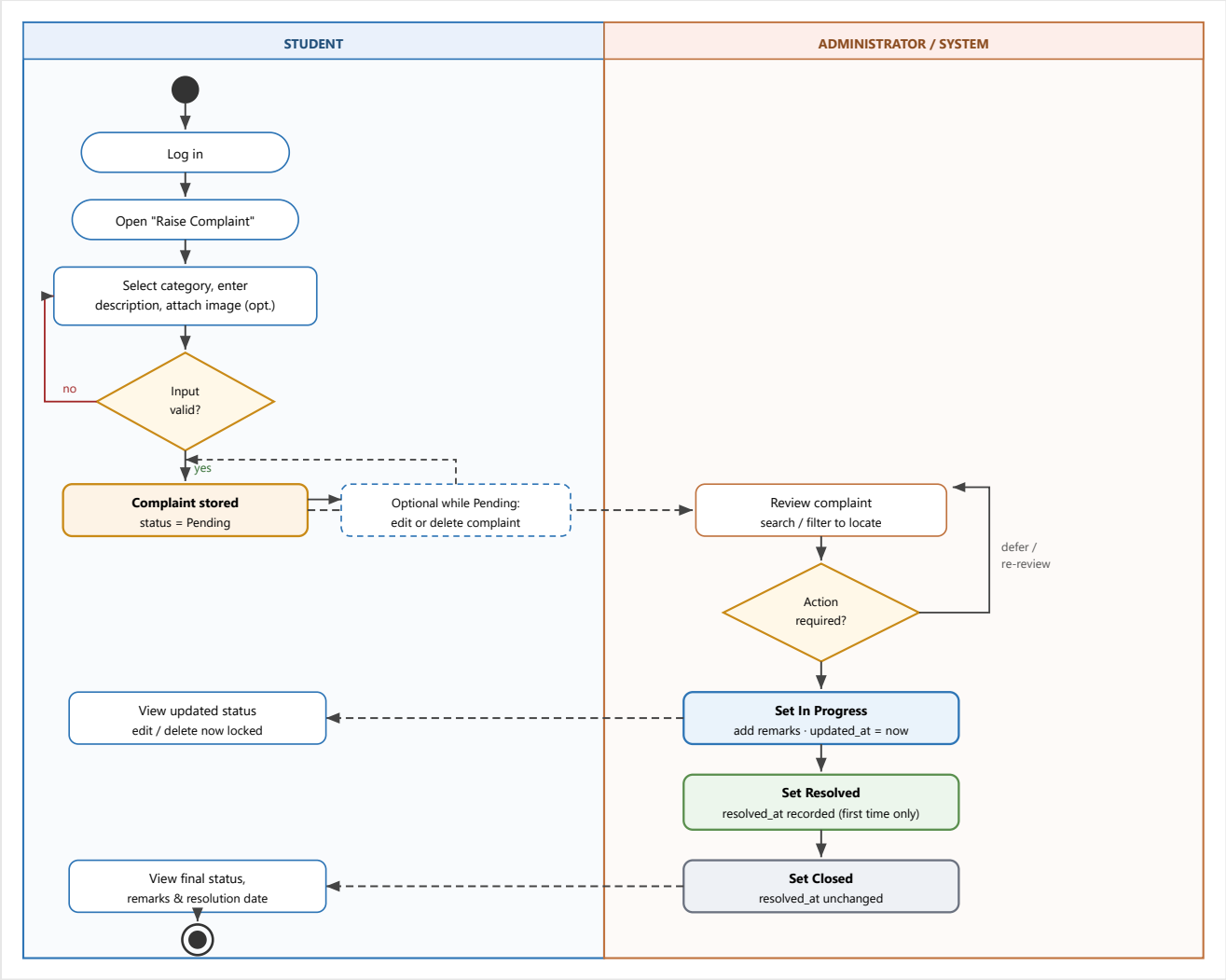

Figure 7.6 — Activity Diagram: end-to-end complaint workflow across both actors.

7.6 State Diagram

The complaint entity is the only object in the system with a non-trivial lifecycle. Its states, permitted transitions and the operations allowed in each state are shown below.

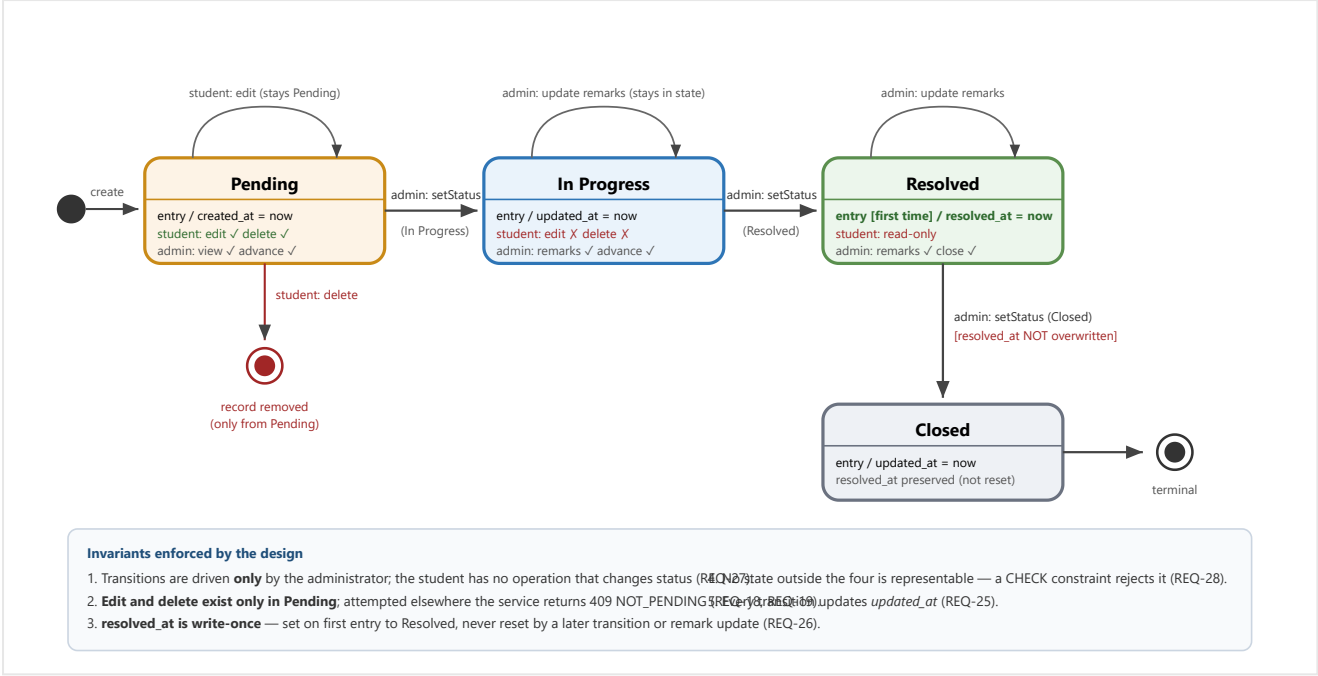

Figure 7.7 — State Diagram for the Complaint entity.

> **Design Rationale.** The lifecycle is modelled as a **forward-only progression** rather than a freely navigable state machine. Backward transitions (for example Resolved → In Progress, to reopen a complaint) were considered and deliberately excluded from v1.0: the SRS defines the workflow as a linear progression, and permitting reversal would immediately raise questions the requirements do not answer — whether `resolved_at` should be cleared, and how a reopened complaint should be counted in the resolution statistics. Excluding them keeps the reporting figures unambiguous. The **write-once `resolved_at`** rule is the single subtlest element of the design and is expressed as a guard on state entry rather than as an unconditional assignment, precisely so that Closed and remark updates cannot falsify the recorded resolution time. Attaching the student's edit/delete permission to the *state* rather than to a separate flag means the permission cannot fall out of step with the status — there is only one source of truth.

8. User Interface Design

8.1 UI Design Principles

Principle	Application in Hostel Buddy
Consistency	One stylesheet, one navigation component and one colour system across all nine pages. A lifecycle state is represented by the same coloured badge everywhere it appears — amber Pending, blue In Progress, green Resolved, grey Closed — so a user learns the vocabulary once. Primary actions always use the same button style and sit in the same position.
Role clarity	The two roles are visually distinguishable at a glance: the student interface uses a navy header, the administrator interface a maroon one. This prevents the common demonstration error of losing track of which account is signed in.
Usability & error prevention	Controls that would fail are **not rendered** rather than shown disabled: Edit and Delete appear only on Pending rows, so a student is never invited to attempt a forbidden action. Required fields are validated on the client for immediate feedback, and a live character counter shows the remaining description budget before submission is attempted.
Feedback	Every asynchronous operation has a visible state: a spinner while loading, a success banner on completion, an inline error banner carrying the server's message on failure, and a disabled button with changed label ("Submitting…") to prevent double submission.
Recognition over recall	Categories are chosen from a dropdown rather than typed; statuses are shown as labelled badges rather than codes; dates are formatted in the reader's locale rather than as raw timestamps.
Progressive disclosure	Lists show only the columns needed to scan and identify a complaint; full detail — description, image, remarks, all timestamps — is revealed in a modal on demand, keeping the table readable.
Responsiveness	Fluid layouts using CSS grid and flexbox; statistic cards reflow from a row to a column, charts stack vertically, and the navigation collapses to a toggle menu on narrow screens. Tables scroll horizontally within their container so the page itself never scrolls sideways.
Accessibility	Semantic HTML elements (headings, tables, labels bound to inputs); text and background combinations chosen for adequate contrast; status conveyed by both colour and text so the interface does not depend on colour perception alone; keyboard-operable forms and controls.
Safety	Irreversible actions require confirmation before proceeding. All values rendered from the server are escaped before insertion into the page, so stored text cannot execute as markup.

8.2 Wireframes

The following figures show the realised interface for the principal screens, consistent with the wireframes approved in the SRS (§3.1).

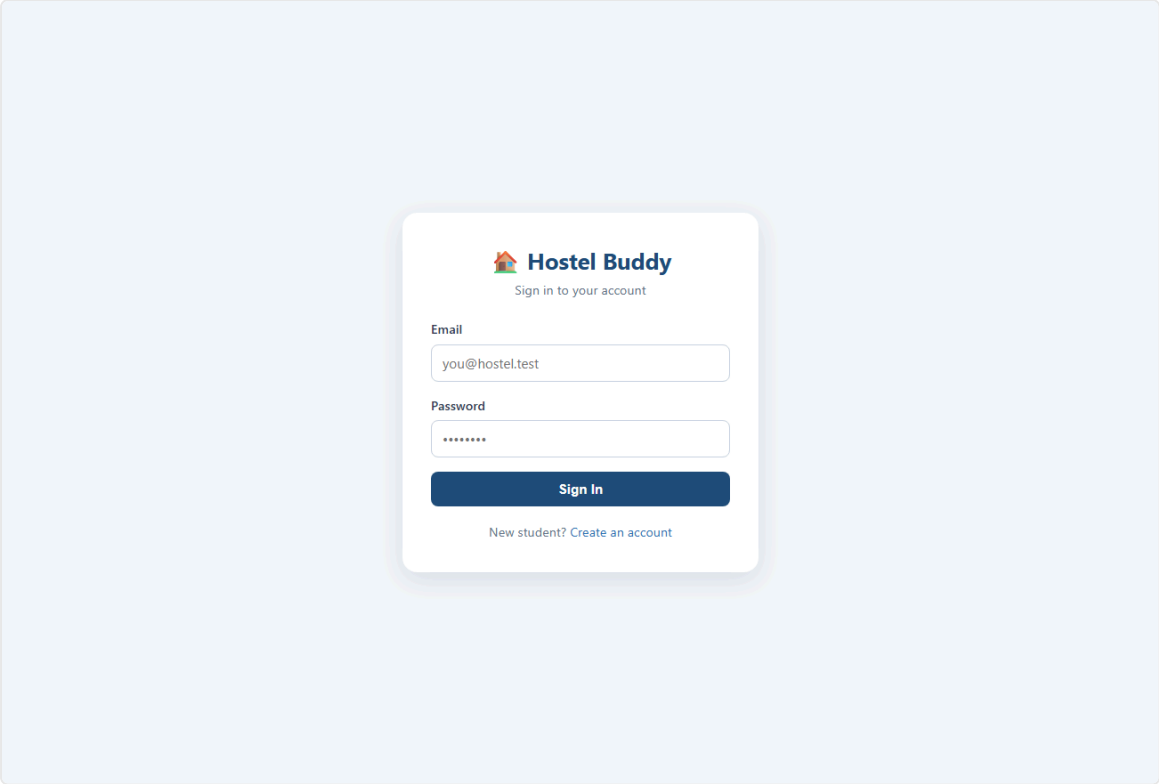

Figure 8.1 — Login (UC-2). A single form serves both roles; the server determines the role and the client routes to the corresponding dashboard.

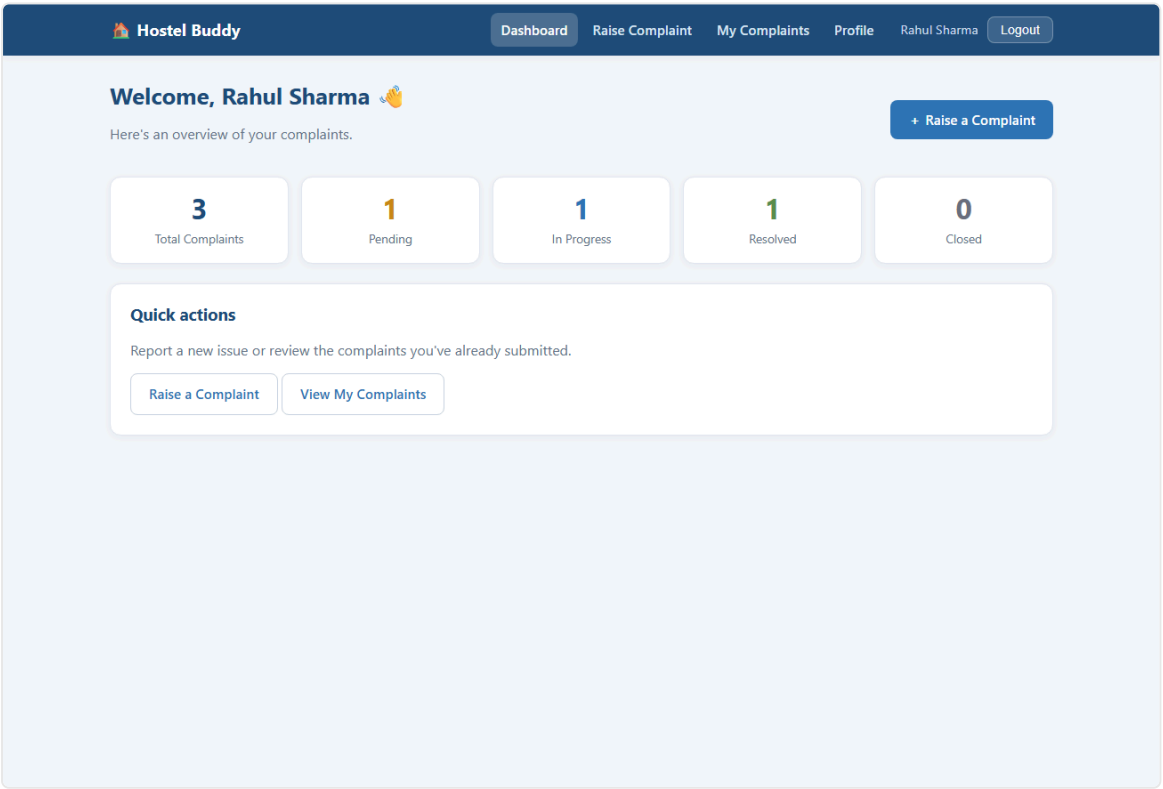

*Figure 8.2 — Student Dashboard (UC-8). Five summary cards realise **REQ-33**; the primary action is placed top-right.*

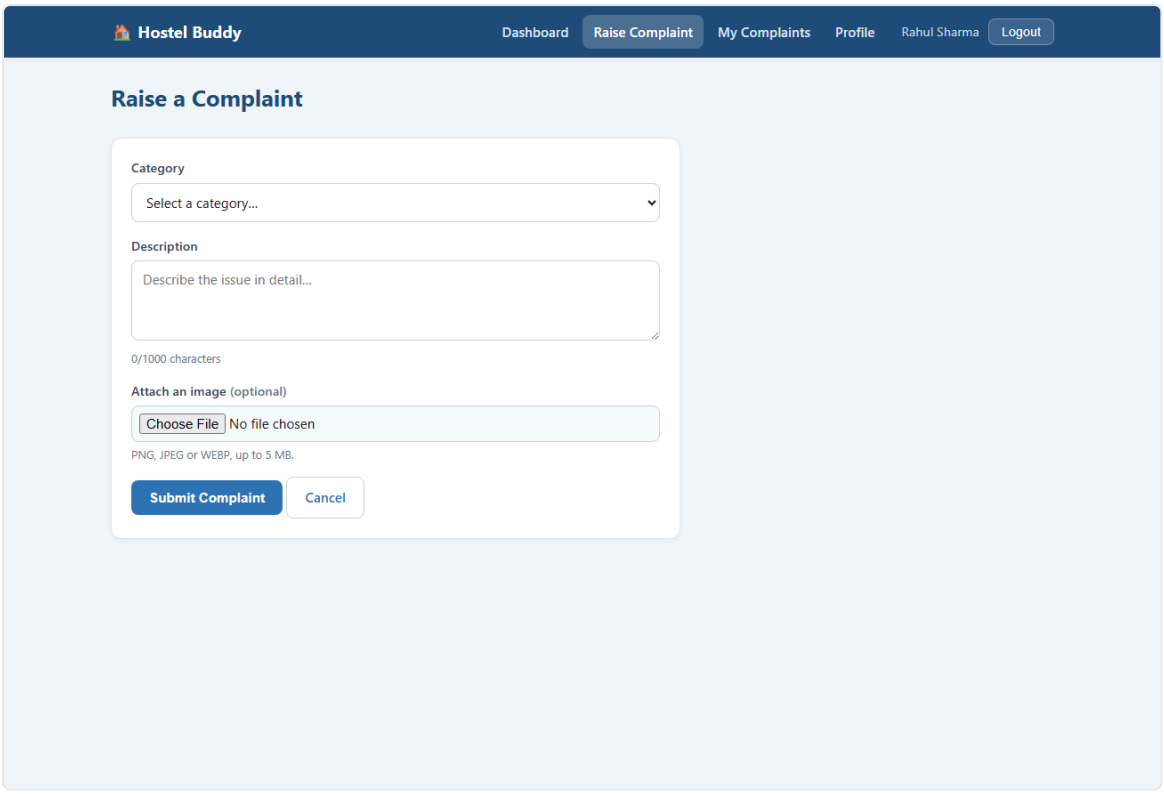

Figure 8.3 — Raise Complaint (UC-4, UC-5). Category dropdown enforces the controlled vocabulary; the image field is explicitly marked optional.

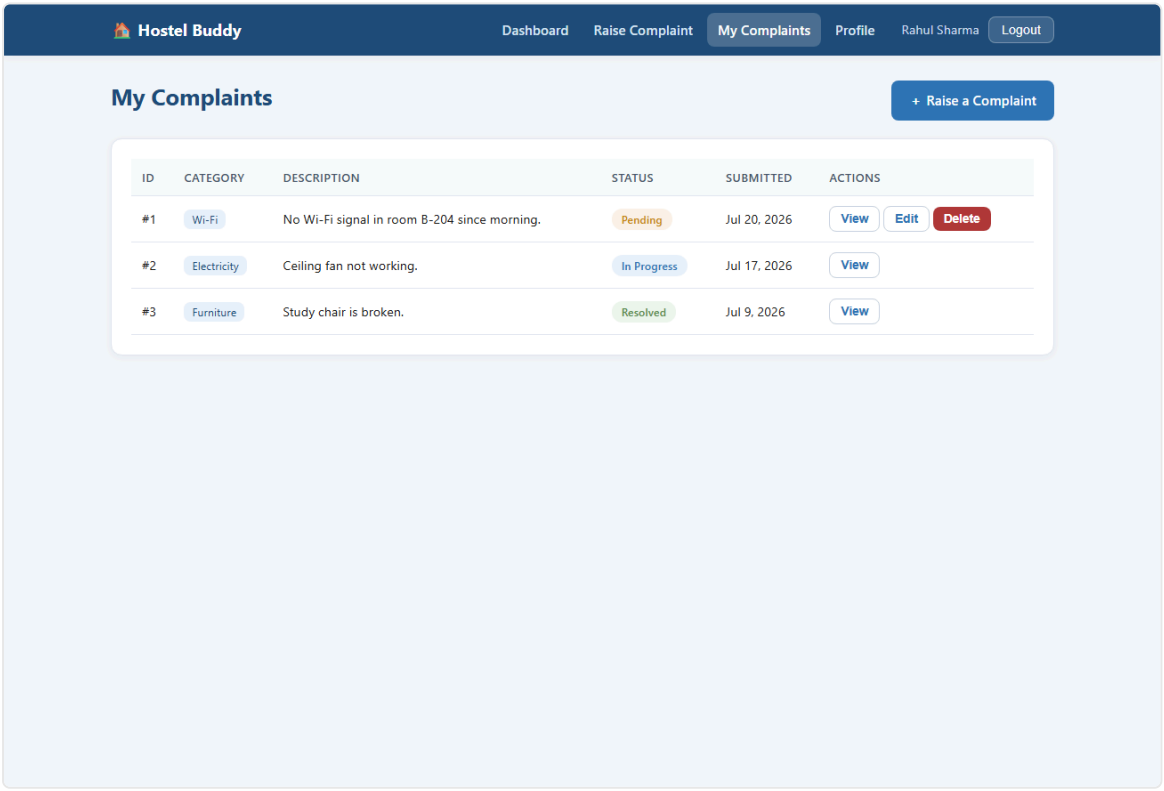

*Figure 8.4 — My Complaints (UC-6, UC-7). Note that **Edit and Delete appear only on the Pending row**, making the rule of **REQ-18**/**REQ-19** visible in the interface itself.*

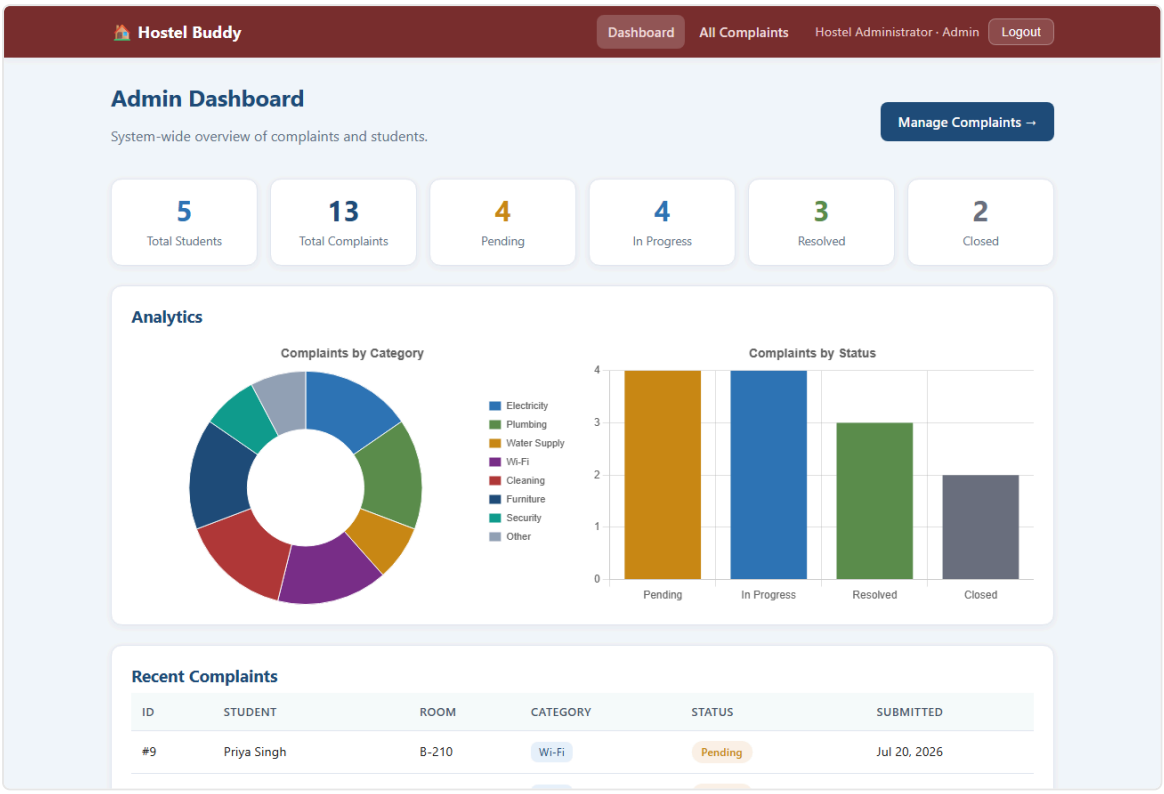

*Figure 8.5 — Administrator Dashboard (UC-13). Statistic cards, category distribution doughnut and status bar chart realise **REQ-34**–**REQ-37**.*

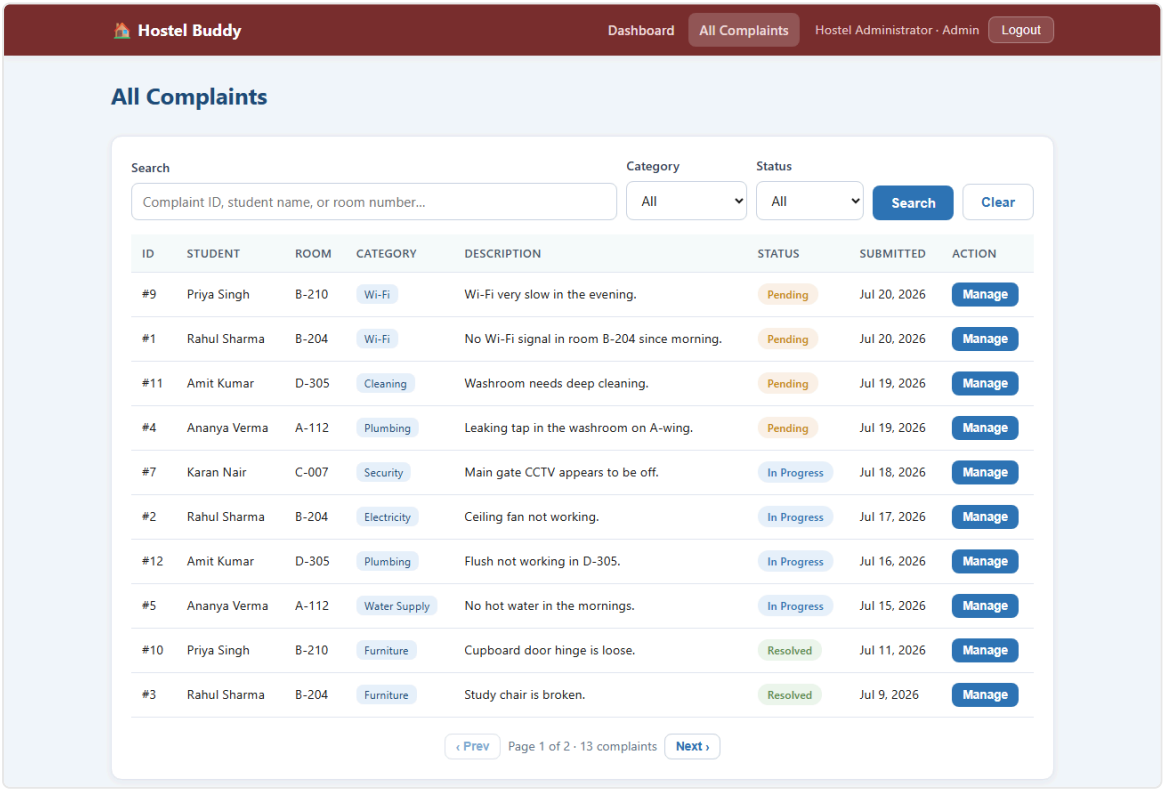

*Figure 8.6 — All Complaints (UC-9, UC-10, UC-11). Search box and two filters realise **REQ-29**–**REQ-32**; the Manage action opens the status/remarks modal.*

8.3 Navigation Flow

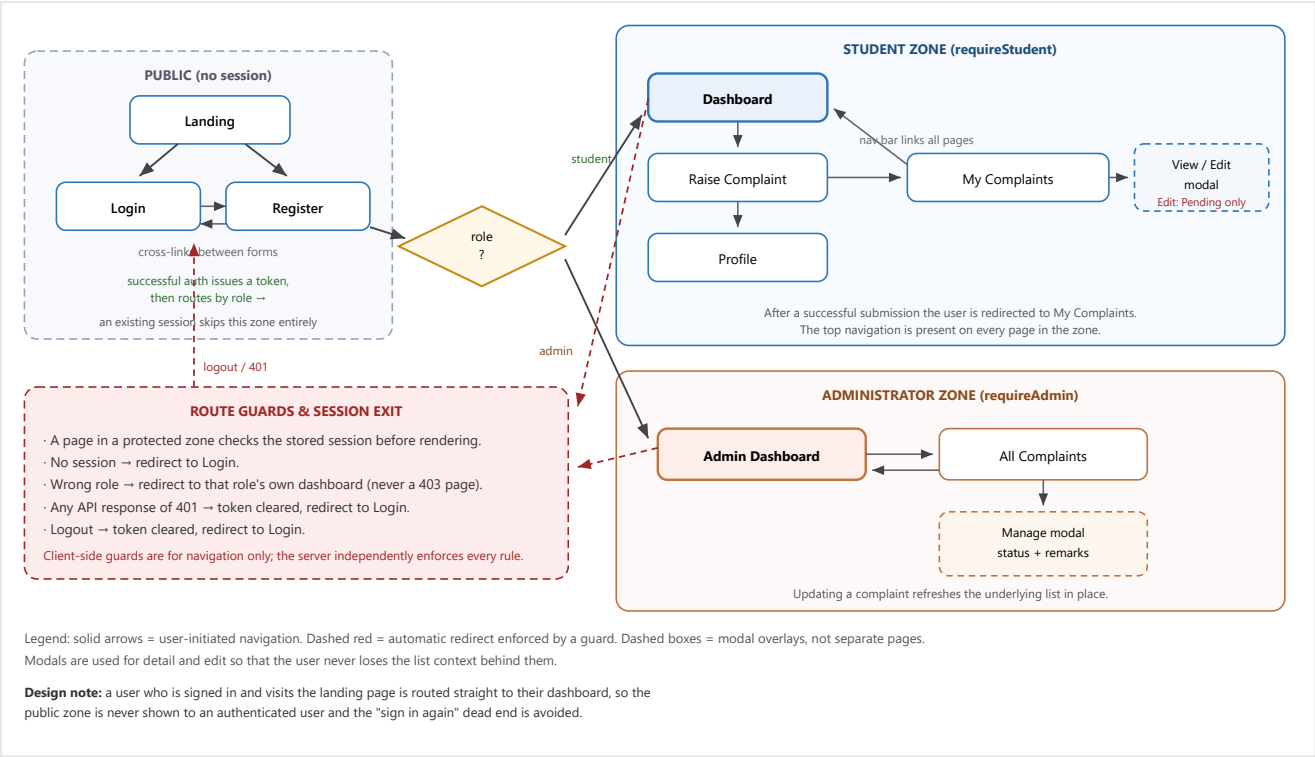

Figure 8.7 — Navigation Flow Diagram with route guards.

> **Design Rationale.** Navigation is partitioned into three **zones** rather than a flat page list, because access control is the property that actually determines where a user may go. Modelling it this way makes the guard rules explicit and reviewable. A user in the wrong zone is **redirected to their own dashboard rather than shown an error page**: the situation is a navigation mistake, not a security incident, and a redirect is a more helpful response than a denial. Detail and edit are presented as **modals rather than separate pages** so the surrounding list is preserved, which matters most on the administrator screen where a filter has usually just been applied and would otherwise be lost. Critically, these client-side guards are treated purely as navigation convenience — every one of them is duplicated as a server-side check, because a determined user can bypass anything the browser enforces.

9. Security Design

9.1 Security Requirements

The security design addresses the requirements of SRS §5.3 (NFR-8–NFR-13). The table states each requirement together with the threat it counters.

Property	Requirement	Principal threats addressed
Authentication	Only a registered user may obtain a session (NFR-13).	Impersonation; unauthenticated access to complaint data; credential brute-forcing; account enumeration.
Authorisation	Role-based access control; a student may reach only their own data (NFR-9, NFR-10).	Privilege escalation (a student performing administrator actions); horizontal access violation (reading or modifying another student's complaint); insecure direct object reference.
Confidentiality	Passwords stored irreversibly; data encrypted in transit (NFR-8, NFR-12).	Credential disclosure following a database compromise; interception of credentials or tokens on the network.
Integrity	All input validated and sanitised; workflow-owned fields not client-settable (NFR-11).	SQL injection; cross-site scripting; workflow bypass by submitting a forged status; malicious file upload.
Availability	Abuse-resistant endpoints and bounded resource use.	Brute-force login floods; oversized uploads; unbounded result sets exhausting memory.
Privacy	Personal data exposed only to its owner and the administrator.	Leakage of student names, rooms and complaint contents to other students.

9.2 Authentication

Mechanism. Credentials are exchanged once for a signed, time-limited token which is then presented on every subsequent request.

- **Registration.** Input is validated, the email normalised to lower case and checked for uniqueness, and the password hashed with bcrypt at cost factor 10 before any storage occurs. The plaintext password exists only for the duration of the request and is never written to the database or to a log.
- **Login.** The submitted password is compared against the stored digest using bcrypt's constant-time comparison. On success a JWT is issued containing only the user identifier and role — no personal data and no secret — signed with HS256 using a server-held secret and expiring after 24 hours.
- **Session verification.** Every protected request carries the token in the `Authorization` header. The authentication middleware verifies the signature and expiry and attaches the resulting identity to the request. A request without a valid token never reaches a controller.
- **Account enumeration resistance.** "Unknown email" and "incorrect password" return an **identical** generic message and status, so the API cannot be used to discover which addresses are registered.
- **Administrator provisioning.** There is no administrator self-registration endpoint. The single administrator is seeded from configuration at start-up, and the seeding operation is idempotent, so restarting the application cannot create a second privileged account.
- **Brute-force mitigation.** Authentication endpoints are rate-limited per client address in production; exceeding the limit returns `429` without revealing whether any attempted credential was valid.

- **Logout.** The client discards the stored token. Because sessions are stateless there is no server record to clear.

Accepted trade-off. A stateless token cannot be revoked before it expires — a token stolen at minute one remains usable until the 24-hour expiry. This was accepted deliberately in exchange for horizontal scalability (§2.5). The mitigations are the bounded lifetime and mandatory HTTPS; a server-side revocation list is the documented upgrade path should the threat model change.

9.3 Authorization

Authorisation is applied in **two independent layers**, so that no single omission can expose data.

Layer	Mechanism	What it protects
Route-level (vertical)	`requireRole('student')` / `requireRole('admin')`	Prevents a role from invoking an operation reserved for the other role — for example a student calling the status-transition endpoint, which is rejected with `403` before any business code executes.
Service-level (horizontal)	Ownership comparison inside the service	Prevents a legitimately authenticated student from acting on *another student's* record. The complaint's stored author is compared with the authenticated caller; a mismatch yields `403` regardless of which route was used.
State-level	Lifecycle guard inside the service	Prevents a legitimate owner from performing an operation that the record's state forbids — editing or deleting a complaint that has left Pending yields `409`.

Permission matrix

Operation	Student (own record)	Student (other's record)	Administrator
Create complaint	✓ permitted	— not applicable	✗ 403 (students only)
View complaint	✓ permitted	✗ 403	✓ permitted (all)
Edit / delete complaint	✓ only while Pending, else 409	✗ 403	✗ 403
List all complaints, search, filter	✗ 403	✗ 403	✓ permitted
Change status / add remarks	✗ 403	✗ 403	✓ permitted
View own profile / dashboard	✓ permitted	✗ 403	✓ own admin dashboard
List registered students	✗ 403	✗ 403	✓ permitted

Why the duplication is deliberate. A route guard protects one route; if a second endpoint were later added for the same operation and the guard forgotten, the rule would silently disappear. Placing the ownership and state rules in the service means they are enforced for every caller, present and future — the reasoning recorded as design decision D1 in Appendix D.

9.4 Data Security

Asset	Protection
Passwords	Stored only as bcrypt digests with a per-password salt and cost factor 10, making offline brute-forcing expensive and precomputed-table attacks ineffective. The digest is never returned by any interface: the repository exposes a public projection omitting it, and only the credential-verification path reads the full row.

Session tokens	Signed with a server-held secret so they cannot be forged or altered; they carry no sensitive payload and expire after 24 hours. The signing secret is supplied by environment configuration and never committed to version control.
Data in transit	HTTPS is required in deployment. Strict-Transport-Security instructs browsers to refuse subsequent plaintext connections to the origin.
Data at rest	The database is a single file whose protection relies on host filesystem permissions; only the application account requires access. Backups must include both the database file and the upload directory to remain consistent.
Uploaded images	Only three image MIME types are accepted and the extension is derived from the verified type rather than from the user-supplied filename, so an executable cannot be stored under an image name. Files are written with a randomly generated name, which also prevents one upload from overwriting another. Size is capped at 5 MB.
Secrets and configuration	All secrets are read from environment variables through a single configuration module. The environment file is excluded from version control and only a placeholder template is committed. In production the application **refuses to start** if the signing secret is missing or left at its development default, or if the administrator password is unchanged — an insecure deployment fails loudly instead of running silently.
Error output	Server faults are logged in full internally but returned to the client as a generic message, so stack traces, file paths and SQL fragments are never disclosed.

Known limitation, consciously accepted. Uploaded images are served from a static path that is not itself behind the ownership check applied to the complaint record. Filenames are long and randomly generated, so URLs are impractical to guess, and they are surfaced only within authorised views. Recorded as decision D4 (Appendix D); the production remedy is to stream images through an authorising route or to issue short-lived signed URLs from object storage.

9.5 Input Validation

The system treats **all client input as untrusted**. Client-side checks exist solely for responsiveness; every rule is re-applied on the server, which is the only authority.

Threat	Countermeasure
SQL injection	Every database operation uses a prepared statement with bound parameters, including the administrator's free-text search. No query is assembled by string concatenation of user input, so input cannot alter query structure. Filter values are additionally checked against the permitted vocabularies before reaching the repository.
Cross-site scripting (XSS)	All server-supplied values are escaped before insertion into the page, so a complaint description containing markup is displayed as text and never executed. A Content Security Policy restricting scripts to the application's own origin provides a second line of defence; achieving this required moving the last inline script into an external file (decision B7, Appendix D).
Workflow bypass	Fields owned by the workflow are never accepted from the client. A complaint's status is forced to Pending on creation regardless of any submitted value, and can subsequently be changed only through the administrator-only transition endpoint.
Mass assignment / privilege escalation	Profile updates apply only an explicit allow-list of fields (name, room number). Role and email are not assignable through any interface, so a user cannot promote themselves to administrator by adding a field to a request.
Malicious file upload	MIME type allow-list, server-derived extension, randomised filename and a size limit; rejected uploads are removed, as are files orphaned when a later validation step fails.
Invalid enumeration values	Category and status are validated against the controlled vocabulary in the service and constrained again by the database schema, so an invalid value cannot be stored even if application validation were bypassed.
Oversized or malformed payloads	Length limits on all text fields; bounded page sizes on listings; malformed JSON is rejected with a validation error rather than propagating as a server fault.

Cross-site request forgery	Structurally mitigated: the session token is held in browser storage and sent explicitly in a header rather than in a cookie, so a third-party site cannot cause an authenticated request to be issued implicitly.

10. Error Handling

10.1 Error Handling Strategy

The system distinguishes **expected failures** — a validation problem, a permission denial, a missing record — from **unexpected faults**, and handles each appropriately. Expected failures are part of the design and produce precise, actionable messages; unexpected faults are logged in full and reported generically.

Mechanism

- **Deliberate failures are thrown as a typed application error** carrying a message, an HTTP status and a machine-readable code. A service that detects a rule violation simply throws; it does not construct an HTTP response, which keeps business logic free of transport concerns.
- **A single central handler** is registered last in the middleware pipeline and converts every error — thrown deliberately or raised unexpectedly — into one uniform response shape. Because it is the only place that formats errors, the contract cannot drift between endpoints.
- **Uniform response envelope:** every failure returns `{ error: { message, code } }`. The client therefore has exactly one error path to implement.
- **Fault masking:** for any status of 500 or above the true message is written to the server log while the client receives a generic apology, so internal details are never disclosed.
- **Fail fast:** input is rejected at the earliest layer competent to judge it — malformed uploads in middleware, business-rule violations in the service — so invalid data never reaches persistence.
- **Resource cleanup:** when a request fails after a file has already been stored, the orphaned file is deleted before the error is returned, so failed submissions cannot accumulate on disk.
- **Startup validation:** configuration is checked when the process starts. In production, insecure settings cause an immediate, explanatory exit rather than a silently vulnerable deployment.

Error catalogue

Status	Code	Raised when	User-visible outcome
`400`	`VALIDATION_ERROR`	Missing or malformed field; invalid category or status; description too long.	Inline message naming the problem, e.g. "Description is required".
`400`	`INVALID_FILE_TYPE`	Upload is not PNG, JPEG or WEBP.	"Only PNG, JPEG, or WEBP images are allowed."
`400`	`FILE_TOO_LARGE`	Upload exceeds 5 MB.	"Image is too large (max 5 MB)."
`401`	`UNAUTHENTICATED`	Absent, malformed or expired token.	Token cleared and user returned to the login screen.
`401`	`INVALID_CREDENTIALS`	Unknown email *or* wrong password.	Deliberately identical generic message for both cases.
`403`	`FORBIDDEN`	Wrong role for the endpoint, or a student acting on another student's record.	"You do not have permission to perform this action."
`404`	`NOT_FOUND`	Referenced complaint or user does not exist; unknown API path.	"Complaint not found."

`409`	`EMAIL_TAKEN`	Registration with an already-registered address, including the concurrent-registration race caught by the database constraint.	"An account with this email already exists."
`409`	`NOT_PENDING`	Attempt to edit or delete a complaint that has left the Pending state.	"This complaint can no longer be edited or deleted because it is already being handled."
`429`	`RATE_LIMITED`	Too many authentication attempts from one address (production).	"Too many attempts. Please try again later."
`500`	`INTERNAL_ERROR`	Any unanticipated fault.	Generic message; full details recorded server-side only.

Client-side handling

The browser client concentrates all error handling in its single network wrapper. It reads the uniform envelope, displays the server's message in the page's alert region, and treats `401` specially by clearing the stored session and returning the user to the login screen. Because every page issues requests through this one component, no page needs its own error logic and behaviour cannot diverge between screens — the reasoning recorded as decision B5 in Appendix D.

Logging and recovery

- Each request is logged with method, path, resulting status and duration, giving a simple audit and performance trail.
- Server faults are logged with their full stack trace; client errors are not, to keep the log signal-to-noise ratio useful.
- A failed request leaves no partial state: writes are single statements, and files stored during a subsequently rejected request are removed.
- Recovery is by retry — all operations except complaint creation are idempotent, so a client may safely repeat a request after a transient failure.

11. Deployment Design

11.1 Deployment Overview

The system is deployed as a **single self-contained Node.js process** that serves both the REST API and the static frontend, backed by a file-based database and a local upload directory. Three environments are defined.

Environment	Purpose	Configuration
Development	Local implementation and debugging by team members.	Runs on `localhost:4000` over HTTP. Verbose request logging; rate limiting disabled so it cannot interfere with rapid iteration or the test suite; database seeded with demonstration data.
Testing	Executing the integration suite against a running instance.	Identical to development but started from a freshly seeded database so results are reproducible. The suite exercises the API exactly as a client would.
Production / Demonstration	Public deployment for classmates and evaluation.	HTTPS terminated by a reverse proxy; strong signing secret and changed administrator password supplied by environment variables — the application refuses to start otherwise; rate limiting active; concise logging; database and uploads on persistent storage.

Scaling path. Because the application tier holds no session state, the single instance can be replicated behind a load balancer without code change. Two components must then be externalised: the database must move to a networked engine such as PostgreSQL (a change confined to the repository layer), and the upload directory must move to shared object storage. This ordering is deliberate — the design keeps both substitutions localised so that scaling is a configuration and infrastructure exercise rather than a rewrite.

11.2 Deployment Diagram

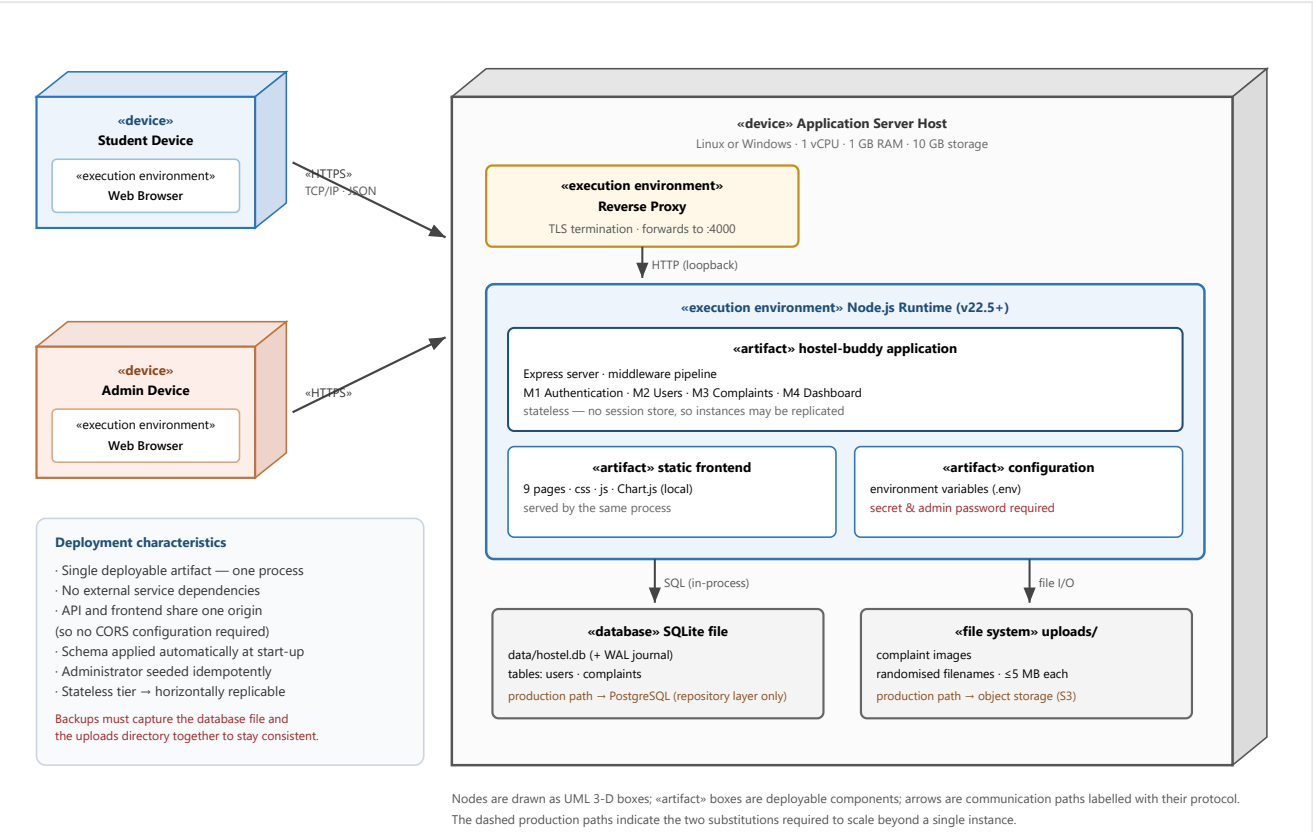

Figure 11.1 — Deployment Diagram.

Design Rationale. A **single-node deployment** was chosen because it matches the actual load — one hostel, a few hundred students, a handful of complaints per day — and because every additional moving part is a component that can fail during a demonstration. Serving the frontend from the same process as the API removes an entire class of problems (cross-origin configuration, a second deployment target, version skew between client and server) at no meaningful cost, since the static assets are a few hundred kilobytes. Placing a **reverse proxy in front for TLS termination** rather than terminating TLS in the application keeps certificate management outside the application lifecycle, so renewing a certificate does not require redeploying the software. The file-based database and local upload directory are the two components that do not survive horizontal scaling; both are deliberately isolated behind narrow interfaces — the repository layer and the upload middleware respectively — so that the migration path is a bounded change rather than an architectural revision. This is the clearest example in the design of the general principle that *the components most likely to change are the ones kept most isolated*.

11.3 Hardware Requirements

Component	Minimum	Recommended (demonstration deployment)
Server CPU	1 vCPU	2 vCPU — the workload is I/O-bound, not compute-bound; bcrypt hashing on login is the only CPU-intensive operation.
Server memory	512 MB	1 GB — the Node process, the in-process database engine and OS overhead fit comfortably.

Server storage	2 GB	10 GB — application and dependencies occupy well under 200 MB; the remainder is headroom for the database and uploaded images (each capped at 5 MB).
Network	Any broadband connection with a public endpoint	Stable connection; payloads are small apart from image uploads.
Client device	Any desktop, laptop, tablet or smartphone able to run a modern browser	No installation, no minimum specification beyond the browser; the interface is responsive down to mobile widths.

The measured query performance (all hot paths at or below approximately one millisecond over a 2,000-complaint dataset) confirms that these modest specifications are appropriate; the system is limited by the single-writer characteristic of the file-based database long before it is limited by CPU or memory.

11.4 Software Requirements

Layer	Software	Notes
Server operating system	Linux, Windows or macOS	No platform-specific dependency; paths are resolved relative to the project root so the same configuration works across platforms.
Runtime	Node.js 22.5 or later	Required for the built-in SQLite module. Development and verification were carried out on Node.js 24.
Package manager	npm 10+	Bundled with Node.js.
Database engine	SQLite (embedded in the runtime)	No separate server process, service account or driver installation. **No native compiler toolchain is required** — a deliberate choice so any team member can build the project.
Application dependencies	express, jsonwebtoken, bcryptjs, multer, helmet, express-rate-limit, dotenv	All pure-JavaScript packages; installed by a single command.
Client-side library	Chart.js 4	Vendored into the repository and served locally, so the application works offline and requires no external origin in its Content Security Policy.
Reverse proxy (production)	Nginx, Caddy or an equivalent	Terminates HTTPS and forwards to the application port.
Client	Chrome, Firefox, Edge or Safari (current versions)	Requires JavaScript and `localStorage`.
Development tooling	Git	Version control; the repository history documents the phase-wise construction.

11.5 Installation Requirements

Installation is a four-step procedure requiring no build step and no compiler.

Step	Action	Detail
1	Obtain the source	Clone the repository. Runtime directories (database, uploads) are excluded from version control and are created automatically.
2	Install dependencies	A single package-install command retrieves the pure-JavaScript dependencies. No native build occurs, so no compiler toolchain is needed.
3	Configure the environment	Copy the committed template to the environment file and set the port, signing secret, administrator credentials and storage paths. In production the signing secret must be strong and the administrator password must be changed, or the application will refuse to start.
4	Start the application	On first start the schema is applied automatically and the administrator account is seeded idempotently. An optional seeding command populates

		sample students and complaints for demonstration.

Post-installation verification

- The health endpoint responds successfully, confirming the process is serving requests.
- The administrator can sign in with the configured credentials.
- The integration suite is executed against the running instance; all checks across the authentication, complaint, administrator and dashboard suites pass.
- The acceptance scenario of SRS §10 is exercised end to end: register, raise a complaint with an image, edit it while Pending, have the administrator advance it through the lifecycle, and confirm the student sees the updated status, remarks and resolution date.

Operational procedures

- **Backup:** copy the database file together with the upload directory; the two must be captured as a set, since complaint records reference image files by path.
- **Upgrade:** stop the process, update the source, reinstall dependencies, restart. Schema additions are applied idempotently at start-up.
- **Log inspection:** request lines carry method, path, status and duration; server faults include a full stack trace for diagnosis.

11.6 Third-party Services

Version 1.0 uses **no third-party services**: no email or SMS gateway, no payment provider, no map or analytics service, no external authentication provider and no cloud storage. This was a deliberate scoping decision — it removes external credentials, network dependencies, cost and privacy considerations from the project, and means the system can be demonstrated with no internet connectivity beyond reaching the host itself.

The third-party **software libraries** used are listed in §11.4; all are open-source packages installed locally and none involves a runtime call to an external service. The client-side charting library is vendored into the repository rather than loaded from a content delivery network, so no request leaves the origin at run time.

Anticipated future service	Purpose	Integration point
SMTP gateway	Email notification on status change	A hook in the complaint service invoked after a successful status transition; requires no change to the API contract.
Push notification provider	Real-time alerts to students	The same post-transition hook, with device registration added to the user record.
Object storage	Durable, shared image storage for multi-instance deployment	Replacement of the upload middleware's storage engine; the surrounding pipeline is unchanged.

12. Traceability Matrix

This matrix demonstrates that every functional requirement of the SRS is realised by identifiable design elements and verified by at least one test. Test-case identifiers refer to the integration suites described in Appendix C (**TC-A** authentication, **TC-C** complaints, **TC-M** administrator management, **TC-D** dashboard), which together comprise 103 automated checks.

12.1 Requirement Mapping

SRS ID	Requirement	Module	Class / Component	Sequence Diagram	Test Case
REQ-1	Register a student account	M1 Authentication	AuthService.register, UserRepository.createUser	Fig. 7.2	TC-A1
REQ-2	Reject duplicate email	M1	AuthService.register; UNIQUE(email) constraint	Fig. 7.2 (alt)	TC-A2
REQ-3	Store passwords protected	M1	AuthService (bcrypt); UserRepository public projection	Fig. 7.2	TC-A3
REQ-4	Authenticate and establish session	M1	AuthService.login, JwtUtil.signToken	Fig. 7.2	TC-A4
REQ-5	Generic invalid-login error	M1	AuthService.login (single error object)	Fig. 7.2 (alt)	TC-A5
REQ-6	Single seeded administrator	M1 / M5	AdminSeeder (idempotent)	—	TC-A11
REQ-7	Logout terminates session	M1	Client Auth.logout; stateless token expiry	—	TC-A7
REQ-8	Validate credential input	M1	AuthService, Validators	Fig. 7.2	TC-A3
REQ-9	Submit a complaint	M3 Complaints	ComplaintService.createComplaint	Fig. 7.3	TC-C1
REQ-10	Restrict to defined categories	M3	ComplaintService.validateCategory; CHECK constraint	Fig. 7.3	TC-C4
REQ-11	Require a description	M3	ComplaintService.validateDescription	Fig. 7.3	TC-C4
REQ-12	Optional image within limits	M3 / M5	UploadMiddleware.uploadImage	Fig. 7.3	TC-C2, TC-C5
REQ-13	Initial state Pending + timestamp	M3	ComplaintService.createComplaint; column default	Fig. 7.3	TC-C1
REQ-14	Associate with submitting student	M3	ComplaintRepository.create (user_id)	Fig. 7.3	TC-C1
REQ-15	Confirm submission	M3	ComplaintController.create (201)	Fig. 7.3	TC-C1
REQ-16	List own complaints	M3	ComplaintService.listMine, Repository.findByUser	Fig. 7.3	TC-C6
REQ-17	Show status, dates, remarks	M3	ComplaintRepository projection; UI list/modal	Fig. 7.4	TC-M11
REQ-18	Edit only while Pending	M3	ComplaintService.loadOwnedPending	Fig. 7.5	TC-C11
REQ-19	Delete only while Pending	M3	ComplaintService.loadOwnedPending	Fig. 7.5	TC-C11, TC-C12

REQ-20	No access to other students' complaints	M3	ComplaintService.getOne / loadOwnedPending	Fig. 7.5	TC-C8
REQ-21	Reflect administrator changes	M3	ComplaintRepository.findByUser (current row)	Fig. 7.4	TC-M11
REQ-22	View all complaints	M3	ComplaintService.listAll, Repository.search	Fig. 7.4	TC-M1
REQ-23	Change status within lifecycle	M3	ComplaintService.updateStatus	Fig. 7.4	TC-M10
REQ-24	Add or update remarks	M3	ComplaintService.updateStatus (remarks policy)	Fig. 7.4	TC-M10, TC-M13
REQ-25	Record last-change time	M3	ComplaintRepository.updateStatus (updated_at)	Fig. 7.4	TC-M10
REQ-26	Record resolution date once	M3	ComplaintService (setResolvedAt guard)	Fig. 7.4	TC-M12, TC-M13
REQ-27	Restrict transitions to administrator	M1 / M3	requireRole('admin') on the transition route	Fig. 7.4	TC-M14
REQ-28	Reject undefined states	M3	ComplaintService.updateStatus; CHECK constraint	Fig. 7.4	TC-M14
REQ-29	Search by ID, name, room	M3	ComplaintRepository.buildFilters / search	Fig. 7.4	TC-M6…M8
REQ-30	Filter by category	M3	ComplaintService.listAll, Repository.search	Fig. 7.4	TC-M4
REQ-31	Filter by status	M3	ComplaintService.listAll, Repository.search	Fig. 7.4	TC-M3
REQ-32	Combine criteria, paginate	M3	ComplaintRepository.search (bound params, LIMIT/OFFSET)	Fig. 7.4	TC-M9
REQ-33	Student statistics	M4 Dashboard	DashboardService.studentDashboard	Fig. 7.4	TC-D1, TC-D5
REQ-34	Total students and complaints	M4	DashboardService.adminDashboard; countStudents	—	TC-D2, TC-D3
REQ-35	Counts per status	M4	ComplaintRepository.statusCounts	—	TC-D2, TC-D6
REQ-36	Category distribution chart	M4	ComplaintRepository.categoryCounts; zeroFilled; Chart.js	—	TC-D2
REQ-37	Recent complaints list	M4	ComplaintRepository.recent	—	TC-D4
REQ-38	View and update own profile	M2 Users	UserService.getProfile / updateProfile	Fig. 7.2	TC-A6, TC-A9
REQ-39	Validate profile changes	M2	UserService.updateProfile (length checks)	—	TC-A9
REQ-40	Administrator views student list	M2	UserService.listStudents; requireRole('admin')	—	TC-A11
REQ-41	Administrator manages accounts	M2	UserRepository (administrator-guarded operations)	—	TC-A10

Non-functional requirement coverage

SRS ID	Requirement	Design element realising it
NFR-1…3	Response-time budgets	Indexes on the three hot paths; aggregation in SQL; bounded pagination. Measured at ≤ ~1 ms per query over 2,000 complaints (§2.1, §6.2).

NFR-4	Scale to thousands of records	Stateless application tier; indexed, paginated queries; documented database substitution path (§11.1).
NFR-5…7	Durability, confirmation of destructive actions, backup	Relational storage with WAL and foreign keys; confirmation before deletion; documented backup procedure (§11.5).
NFR-8	Password hashing	bcrypt cost 10; digest never returned (§9.4).
NFR-9, NFR-10	Role-based and per-owner access control	Route guards plus service-level ownership checks (§9.3).
NFR-11	Input validation and sanitisation	Service validation, prepared statements, output escaping, CSP (§9.5).
NFR-12	Secure transport	HTTPS via reverse proxy; HSTS header (§9.4, §11.2).
NFR-13	Authentication required for data access	requireAuth on every protected route (§9.2).

Coverage summary. All 41 functional requirements and all 13 non-functional requirements of the SRS are traceable to design elements; 39 of the 41 functional requirements are additionally covered by at least one automated test, the exceptions being `REQ-7` (client-side session discard, verified manually) and `REQ-41` (administrator account management, partially implemented as the student listing in v1.0).

13. Assumptions and Limitations

13.1 Assumptions

1. **Single hostel, single administrator.** One privileged account manages all complaints. The data model has no hostel or block dimension.
2. **Trusted administrator.** The administrator is assumed to act in good faith; the design protects students from one another and from unauthenticated access, not from the administrator.
3. **Valid, unique email per student.** Email is the login identifier. Addresses are not verified by a confirmation link in v1.0, so a student may register with an address they do not control.
4. **Modern browser with JavaScript and local storage.** The client is a JavaScript application; it does not degrade to a non-scripted mode.
5. **Moderate concurrency.** A few hundred students with occasional complaints. Write concurrency stays well within the capacity of a single-writer embedded database.
6. **Small images.** Complaint photographs are a few megabytes at most; no server-side resizing or thumbnailing is performed.
7. **Reliable host.** The server and its storage are available during hostel operating hours; no clustering or automatic failover is provided.
8. **Deployment behind TLS.** Any real deployment terminates HTTPS in front of the application, as the design depends on this for transport confidentiality.
9. **Server-controlled clock.** All timestamps are generated by the database on the server; client clocks are neither trusted nor used.
10. **Stable categories.** The eight complaint categories are fixed for v1.0 and are not administrator-editable.

13.2 Constraints

Constraint	Consequence for the design	Mitigation
Academic timeline and team size	Scope was fixed to the SRS baseline; notifications, staff roles and multi-hostel support were excluded.	Extension points documented in §13.3 so deferred features attach without redesign.
No paid infrastructure	No managed database, object storage, mail gateway or push service.	Self-contained deployment; substitution paths recorded for each (§11.6).
No native build toolchain on team machines	Ruled out database drivers requiring compilation, which drove the choice of the runtime's built-in engine.	Data access isolated in repositories, so the engine can be replaced without touching business logic.
Single-writer embedded database	Concurrent writes serialise; this is the first ceiling the system would meet under load.	Documented migration to a networked engine, confined to the repository layer.
Local filesystem storage	Uploaded images are not shared between application instances, blocking horizontal scaling until addressed.	Upload storage isolated behind one middleware component.
Stateless tokens	A session cannot be revoked before its expiry.	Short 24-hour lifetime; revocation list identified as the upgrade path.
Unauthenticated static image path	Complaint images are retrievable by anyone holding the exact URL.	Long random filenames; documented as decision D4 with the production remedy

		specified.
No build pipeline for the frontend	Assets are served unbundled and unminified.	Total payload is small; bundling is a deployment optimisation, not a design change.
English-only interface	No localisation infrastructure in v1.0.	User-facing strings are confined to the presentation layer, so extraction for translation is straightforward.

13.3 Future Enhancements

Each deferred feature is listed with the specific extension point at which it attaches, demonstrating that the architecture anticipates it.

Enhancement	Extension point	Design impact
Email / push notifications	A hook in the complaint service invoked after a successful status transition.	Additive. Requires a notification service in M5 and a delivery worker; the API contract and all existing modules are unchanged.
Maintenance-staff role	The role vocabulary in the constants module, plus an assignment column on the complaint entity.	Moderate. Adds a third role to the permission matrix and an assignment step to the lifecycle; the guard mechanism itself already supports arbitrary roles.
Complaint status history / audit trail	A new history table written by the single transition method in the complaint service.	Low risk. Because every transition passes through one method, capturing history requires one insertion at one place.
Complaint ratings	Additional columns on the complaint entity, written by a new student-only service operation permitted in the Resolved state.	Additive, guarded by the existing state machine.
Administrator-managed categories	Promotion of the category vocabulary from a constant plus CHECK constraint to a lookup table with a foreign key.	Confined to the constants module, the schema and the complaint repository.
Multi-hostel support	A hostel entity referenced by users and complaints, with the identifier added to the authorisation context.	The most invasive change: every query gains a scoping predicate. The centralised repository layer keeps it mechanical.
Native mobile application	The existing REST API, consumed by a new client.	None on the server — the client-agnostic API was the primary motivation for the layered style (§2.2).
AI-based complaint categorisation	A classification step inside the create operation of the complaint service, suggesting a category from the description.	Additive; the category remains user-confirmable, so the controlled vocabulary and its constraint are unaffected.
QR-code room identification	A pre-fill parameter on the raise-complaint screen resolving to a room number.	Presentation-tier only.
Horizontal scaling	Repository layer (networked database) and upload middleware (object storage).	Two bounded substitutions; the application tier is already stateless and requires no change.

Submission Checklist

Item	Completed	Location in this document
Architecture Diagram	☑ Yes	Figure 2.1 (§2.2.1)
Module Decomposition	☑ Yes	Figure 4.1 (§4.3)
Module Interaction	☑ Yes	Figure 4.5 (§4.5)
Database Design	☑ Yes	§6.1–§6.6
ER Diagram	☑ Yes	Figure 6.2 (§6.2)
Data Dictionary	☑ Yes	§6.4
Class Diagram	☑ Yes	Figure 7.1 (§7.2)
Sequence Diagrams (minimum 3)	☑ Yes — 4 provided	Figures 7.2, 7.3, 7.4, 7.5 (§7.4)
Deployment Diagram	☑ Yes	Figure 11.1 (§11.2)
Traceability Matrix	☑ Yes	§12.1 — all 41 functional requirements

Additional diagrams beyond the required set: System Context (Figure 3.1), High-Level Data Flow (Figure 3.2), Package Diagram (Figure 4.6), Activity Diagram (Figure 7.6), State Diagram (Figure 7.7), Navigation Flow (Figure 8.7) and six interface figures (§8.2).

Appendix A — Glossary

Term	Definition
Complaint	A maintenance issue reported by a student, comprising a category, description, optional image, lifecycle status, administrator remarks and timestamps.
Category	The type of issue, drawn from a fixed set of eight values.
Lifecycle / Status	The stage of a complaint: Pending, In Progress, Resolved or Closed.
Pending	A newly submitted complaint not yet acted upon; the only state in which its author may edit or delete it.
In Progress	A complaint the administrator has begun handling; locked to the student.
Resolved	A complaint whose issue has been fixed; the moment of first reaching this state is recorded permanently.
Closed	A resolved complaint finalised with no further action required.
Remarks	Notes recorded by the administrator and visible to the complaint's author.
Module	A cohesive unit of the system owning one major functional responsibility (§4.1).
Layer	A horizontal division within a module: routes, controller, service, repository.
Service layer	The layer holding all business rules; the authority boundary of the system.
Repository	The only component permitted to execute SQL against a given table.
Middleware	A pipeline function processing a request before it reaches a controller.
Guard	Middleware or a service check that rejects an unauthorised request.
Read-model	A projection assembled purely for display, owning no storage (the Dashboard module).
Stateless	Holding no per-user server-side session, so any instance can serve any request.
Idempotent	An operation producing the same result when repeated, e.g. administrator seeding.

Write-once field	A column set at most once in a record's lifetime; here, the resolution timestamp.
Defence in depth	Enforcing a rule at more than one layer so a single omission is not sufficient to breach it.
JWT	JSON Web Token — a signed token carrying user identity and role.
bcrypt	Adaptive password-hashing function with a configurable work factor.
RBAC	Role-Based Access Control.
CSP	Content Security Policy restricting which resources a page may load.
WAL	Write-Ahead Logging, a journalling mode improving read concurrency.
3NF	Third Normal Form — the normalisation level of this schema (§6.6).

Appendix B — API Specifications

Base path `/api`. All protected endpoints require the header `Authorization: Bearer <token>`. Failures return `{ error: { message, code } }`.

Method	Path	Access	Request	Success response
GET	`/health`	Public	—	`200` service liveness indicator
POST	`/auth/register`	Public	name, email, password, room_number	`201` { token, user }
POST	`/auth/login`	Public	email, password	`200` { token, user }
GET	`/users/me`	Authenticated	—	`200` user (no password hash)
PUT	`/users/me`	Authenticated	name, room_number	`200` updated user
GET	`/users`	Administrator	—	`200` array of students
POST	`/complaints`	Student	multipart: category, description, image?	`201` complaint (status Pending)
GET	`/complaints/mine`	Student	—	`200` array, newest first
GET	`/complaints/:id`	Owner or Administrator	—	`200` complaint
PUT	`/complaints/:id`	Student (owner, Pending)	multipart: category, description, image?	`200` updated complaint
DELETE	`/complaints/:id`	Student (owner, Pending)	—	`200` { deleted, id }
GET	`/complaints`	Administrator	q, category, status, page, limit	`200` { data[], pagination }
PATCH	`/complaints/:id/status`	Administrator	status, admin_remarks?	`200` updated complaint
GET	`/dashboard/student`	Student	—	`200` counts by state
GET	`/dashboard/admin`	Administrator	—	`200` totals, byStatus, byCategory, recent[]
GET	`/uploads/:file`	Public (see D4)	—	`200` image bytes

Appendix C — Additional UML Diagrams and Verification

C.1 Diagram index

Figure	Diagram	Section
Figure 2.1	Layered architecture	§2.2.1
Figure 3.1	System context	§3.1
Figure 3.2	High-level data flow	§3.3
Figure 4.1	Module decomposition	§4.3
Figure 4.5	Module interaction	§4.5
Figure 4.6	Package diagram	§4.6
Figure 6.2	Entity–relationship model	§6.2
Figure 7.1	Class diagram	§7.2
Figures 7.2–7.5	Sequence diagrams (registration, complaint submission, resolution, rejected edit)	§7.4
Figure 7.6	Activity diagram	§7.5
Figure 7.7	State diagram	§7.6
Figures 8.1–8.6	Interface screens	§8.2
Figure 8.7	Navigation flow	§8.3
Figure 11.1	Deployment diagram	§11.2

C.2 Verification suites referenced by the traceability matrix

Prefix	Suite	Checks	Principal coverage
TC-A	Authentication & accounts	24	Registration, duplicate rejection, password protection, login, generic error, token guards, role guard, profile read/update, administrator listing.
TC-C	Complaint core (student)	24	Creation with and without image, validation, upload type rejection, own-list retrieval, cross-student denial, edit and delete permitted only while Pending.
TC-M	Administrator management	32	Full listing, search by identifier/name/room, category and status filters, invalid filter rejection, pagination, full lifecycle transition, write-once resolution timestamp, role denial.
TC-D	Dashboard & analytics	23	Student and administrator statistics, zero-filled distributions, aggregate consistency invariants, recent ordering, role separation.
Total		**103**	All suites are self-contained and order-independent, so they may be executed individually or together against a running instance.

Appendix D — Design Decisions

The significant decisions taken during design and construction, each with its alternatives and the trade-off accepted. Identifiers correspond to the project's engineering log.

ID	Decision	Rationale and accepted trade-off
D1	Enforce the ownership and Pending-only rules in the	

	service layer, not in route handlers.	A route-level check protects a single route; if another entry point to the same operation were added later, the rule would silently vanish. Placing the invariant with the business logic guarantees it for every caller. Trade-off: the check is fractionally further from the request boundary.
D2	Stateless JWT sessions rather than server-side session storage.	Removes shared state, so instances replicate without affinity — directly serving the scalability requirement. Trade-off: tokens cannot be revoked before expiry; mitigated by a 24-hour lifetime, with a revocation list as the documented upgrade.
D3	Enforce controlled vocabularies in **both** the application and the database.	Application validation produces helpful messages; the database constraint guarantees correctness even if application code is bypassed by a script or a future bug. Trade-off: a category change requires editing two places, which is acceptable for values fixed by requirement.
D4	Serve complaint images from an unauthenticated static path.	Filenames are long and randomly generated, and URLs appear only inside authorised views, so guessing is impractical. Fully gating images would require streaming every file through an authorising route for little practical gain at this scale. Trade-off explicitly recorded; production remedy is an authorising route or signed object-storage URLs.
D5	Use the runtime's built-in SQLite driver instead of a native-compiled package.	The native package requires a C++ toolchain that was unavailable on team machines, blocking installation entirely. The built-in driver offers the same synchronous prepared-statement interface with no build step, so any member can clone and run the project. Trade-off: the module is marked experimental; isolation within the repository layer makes substitution inexpensive.
B3	Make the resolution timestamp **write-once**.	An unconditional assignment on every save meant a later remark edit, or the transition to Closed, silently overwrote the true resolution time. Expressing it as a guarded state-entry action fixed the defect and preserves the integrity of resolution statistics. Lesson recorded: "the first time X happens" is a distinct rule from "whenever X is true".
B4	Never accept workflow-owned fields from the client.	A creation request could otherwise submit a status and bypass the lifecycle. The service now ignores any client-supplied status and forces Pending; status changes only through the administrator-only transition endpoint.
B5	Centralise all client network access in one wrapper.	Scattered request code duplicated token handling and left unauthenticated responses producing blank screens. A single wrapper attaches the token, applies headers and handles session expiry uniformly, so no page carries its own cross-cutting logic.
B6	Make every test suite self-contained and order-independent.	Suites that assumed pristine seed data passed in isolation but failed when run after others — a test that only passes first is not passing. Each suite now creates its own data and asserts invariants and deltas rather than absolute counts.
B7	Move the last inline script into an external file to permit a strict CSP.	A single inline script would have forced the policy to be weakened for all scripts. Externalising it allows scripts to be restricted to the application's own origin, materially reducing cross-site scripting risk. Lesson recorded: keep scripts external from the outset.
D6	Package source by feature first, layer second.	Layer-first packaging scatters a single change across four distant directories. Feature-first keeps everything needed for one requirement together, reducing navigation cost and merge conflicts on a shared codebase.
D7	Model the lifecycle as forward-only, with no reopening.	Backward transitions would raise questions the requirements do not answer (whether to clear the resolution time; how to count a reopened complaint in statistics). Excluding them keeps reporting unambiguous. Recorded as a candidate enhancement should reopening become a requirement.

Appendix E — Version History

Version	Date	Author	Summary of changes
1.0	5 August 2026	Group 19	Initial Software Architecture and Design Document derived from the approved SRS v1.0. Establishes the layered three-tier architecture, five-module decomposition, relational database design in 3NF, detailed class and interaction design, security and error-handling strategies, single-node deployment

			model, and full traceability of all 41 functional requirements.

Relationship to the Software Requirements Specification

Document	Version	Relationship
Software Requirements Specification	1.0	Defines *what* the system must do. This SADD is derived from it; every requirement it states is traced to a design element in §12.
Software Architecture and Design Document	1.0 (this document)	Defines *how* those requirements are satisfied. Supersedes no part of the SRS; where the two are read together, the SRS remains authoritative on required behaviour.

This document will be revised if the SRS baseline changes or if review feedback requires architectural amendment. Design decisions superseded in a later revision will be retained in Appendix D with their outcome recorded, so the reasoning history remains available.